

Laboratorio di Intelligenza Artificiale

Appunti di corso – A.A. 2025/2026

Prof. Giacomo Fiumara – Università di Messina

Testi di riferimento:

Russell & Norvig, *Artificial Intelligence: A Modern Approach*

A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*

Appunti basati sul materiale di Aurora Fabio

Indice

1	Introduzione all'IA	7
1.1	Che cos'è l'Intelligenza Artificiale?	7
1.1.1	Approccio 1 – Acting Humanly: il Test di Turing	8
1.1.2	Approccio 2 – Thinking Humanly: la modellazione cognitiva	8
1.1.3	Approccio 3 – Thinking Rationally: le leggi del pensiero	8
1.1.4	Approccio 4 – Acting Rationally: l'agente razionale	9
1.2	Fondamenti dell'Intelligenza Artificiale	9
1.2.1	Filosofia	9
1.2.2	Matematica	9
1.2.3	Economia	10
1.2.4	Neuroscienze	10
1.2.5	Psicologia	10
1.2.6	Ingegneria informatica	10
1.2.7	Teoria del controllo e cibernetica	11
1.2.8	Linguistica	11
1.3	Storia dell'Intelligenza Artificiale	11
1.3.1	Le origini (1943–1956)	11
1.3.2	Entusiasmo e aspettative (1952–1969)	11
1.3.3	Una dose di realtà e i primi inverni (1966–1986)	12
1.3.4	Rinascita e Deep Learning (1986–oggi)	12
1.4	Stato dell'arte	12
1.5	Rischi e benefici dell'IA	13
1.5.1	Il Problema dell'Allineamento dei Valori	13
1.5.2	Rischi e benefici	13
2	Agenti Intelligenti	15
2.1	Agenti e interazione con l'ambiente	15
2.1.1	Percetti, sequenza percettiva e funzione agente	16

2.1.2	Il mondo dell'aspirapolvere	16
2.2	Razionalità e misure di prestazione	17
2.2.1	I 4 pilastri della razionalità	17
2.2.2	Razionalità vs. onniscienza e autonomia	17
2.3	La natura degli ambienti: PEAS	18
2.3.1	Dimensioni di classificazione degli ambienti	19
2.4	Struttura e tipologie degli agenti	20
2.4.1	1. Agenti Reflex Semplici	20
2.4.2	2. Agenti Reflex Basati su Modello	21
2.4.3	3. Agenti Basati sull'Obiettivo	23
2.4.4	4. Agenti Basati sull'Utilità	23
2.4.5	5. Agenti che Apprendono	24
2.4.6	Rappresentazioni dello stato	25
3	Introduzione al Machine Learning	26
3.1	Cos'è il Machine Learning?	26
3.2	Come funziona l'apprendimento	26
3.3	Approccio classico vs. Machine Learning	27
3.4	Training set e Test set	28
3.5	La Matrice di Confusione	28
3.6	Struttura di un Dataset	29
3.7	Famiglie di Machine Learning	29
3.7.1	Classificazione per supervisione	29
3.7.2	Batch vs. Online Learning	30
3.7.3	Instance-Based vs. Model-Based Learning	31
3.8	La Regressione Lineare	31
4	Dati, Modelli e i Loro Limiti	32
4.1	Ricapitolo: la regressione lineare e la matematica del modello	32
4.1.1	Come si trovano i parametri ottimali?	33
4.1.2	E se il problema NON è lineare?	33
4.2	Le 4 fasi di un progetto di Machine Learning	34
4.3	I dati: quantità, costo e rappresentatività	34
4.3.1	Quantità e costo computazionale	34
4.3.2	Rappresentatività del campione	35
4.4	I limiti dei modelli: il problema della non-linearità	35
4.5	Overfitting vs. Underfitting	36

4.6	Divisione del dataset e Cross-Validation	37
4.6.1	Suddivisione standard (2 vie)	37
4.6.2	Suddivisione industriale (3 vie)	37
4.6.3	Cross-Validation (Convalida Incrociata)	37
4.7	Iperparametri vs. parametri	38
4.8	No Free Lunch Theorem	38
4.9	Conclusioni sul Machine Learning	38
5	La Classificazione	39
5.1	Introduzione alla classificazione	39
5.2	I dataset storici – le “Baseline”	39
5.2.1	Iris Dataset (Fisher, 1936)	39
5.2.2	MNIST Dataset (LeCun et al., 1998)	40
5.3	Rappresentatività e campionamento stratificato	40
5.4	Addestrare un classificatore binario	41
5.5	Metriche di qualità	41
5.5.1	Perché l’accuratezza da sola non basta	41
5.5.2	Matrice di confusione	41
5.5.3	Precisione	42
5.5.4	Recall (Sensibilità)	42
5.5.5	Il trade-off Precisione – Recall	42
5.5.6	F1-Score	43
6	Trade-off P/R, Curva ROC e Class. Multiclasse	44
6.1	Ricapitolo: il compromesso Precisione – Recall	44
6.2	La curva Precisione-Recall e come trovare la soglia ottimale	45
6.3	La curva ROC e l’AUC	45
6.3.1	Quando usare PR curve e quando ROC	46
6.4	Classificazione Multiclasse	47
6.4.1	One-vs-Rest (OvR)	47
6.4.2	One-vs-One (OvO)	47
6.5	Analisi degli errori	47
6.5.1	Matrice di confusione multiclasse, normalizzata	48
6.5.2	Il trucco della diagonale	48
6.5.3	Feature Engineering mirata	48
6.6	Classificazione Multilabel	49
6.6.1	ClassifierChain: modellare le dipendenze tra label	49

6.7	Classificazione Multioutput	49
7	Alberi di Decisione	50
7.1	Introduzione	50
7.2	Come funziona – l'esempio Iris	50
7.2.1	I confini di decisione	51
7.3	Metriche di impurità	52
7.3.1	Indice di Gini	52
7.3.2	Entropia di Shannon	53
7.4	L'algoritmo CART	53
7.4.1	CART è un algoritmo greedy	53
7.4.2	Costo computazionale	54
7.5	Regressione con gli alberi di decisione	54
7.6	Overfitting e regolarizzazione	55
7.7	Sensibilità all'orientamento degli assi	55
7.8	Alta varianza e Random Forest	55
8	Apprendimento Non Supervisionato e K-Means	57
8.1	Introduzione	57
8.1.1	Applicazioni principali	57
8.2	Classificazione vs. Clustering	58
8.3	L'algoritmo K-Means	58
8.3.1	Funzionamento passo per passo	58
8.3.2	Tassellazione di Voronoi	59
8.3.3	Hard clustering vs. Soft clustering	59
8.4	Inizializzazione dei centroidi e K-Means++	59
8.4.1	Mini-Batch K-Means	60
8.5	Scegliere il numero ottimale di cluster k	60
8.5.1	Metodo del Gomito (Elbow Method)	60
8.5.2	Coefficiente di Silhouette	60
8.5.3	Diagramma a coltello (Silhouette Diagram)	61
8.6	Applicazioni del K-Means	62
8.6.1	Segmentazione di immagini	62
8.6.2	Apprendimento semi-supervisionato con K-Means	62
8.7	Limiti del K-Means	62
9	DBSCAN, Isolation Forest e Anomaly Detection	64

9.1	DBSCAN: clustering basato sulla densità	64
9.1.1	Concetti chiave	64
9.1.2	Funzionamento dell'algoritmo	64
9.1.3	Vantaggi e limiti	65
9.2	Cenno: Gaussian Mixture Models (GMM)	65
9.3	Isolation Forest: rilevazione delle anomalie	65
9.3.1	Logica fondamentale	66
9.3.2	Come funziona	66
9.3.3	Parametri fondamentali	66
9.3.4	Vantaggi e limiti	66
9.4	Analisi esplorativa (EDA) e outlier	67
9.4.1	Istogrammi e il fenomeno del Capping	67
9.4.2	Il Boxplot (diagramma a scatola e baffi)	67
9.5	Sintesi degli algoritmi non supervisionati	68
10	Reti Neurali Artificiali	69
10.1	Introduzione e applicazioni moderne	69
10.2	Dal neurone biologico al modello matematico	69
10.2.1	McCulloch & Pitts (1943)	70
10.3	Il Percettrone	70
10.3.1	Matematica del percettrone	70
10.3.2	Funzione di attivazione Heaviside	71
10.3.3	Regola di apprendimento di Hebb	71
10.3.4	Limiti del percettrone e l'Inverno dell'IA	71
10.4	Architettura del Multi-Layer Perceptron (MLP)	72
10.5	Funzioni di attivazione	73
10.6	Backpropagation	73
10.6.1	Le quattro fasi	74
10.7	Regressione e classificazione con MLP	74
10.8	Fine-tuning degli iperparametri	75
10.9	Teorema di approssimazione universale	75
11	Addestrare Reti Neurali Profonde	76
11.1	Problemi nell'addestramento di reti profonde	76
11.2	Il problema del vanishing/exploding gradient	76
11.3	Inizializzazione di Glorot e He	77
11.4	Funzioni di attivazione moderne	77

11.5 Gradient Clipping	78
11.6 Ottimizzatori: Momentum	78
11.6.1 Momentum	78
11.6.2 Nesterov Accelerated Gradient (NAG)	78
11.7 Tecniche di regolarizzazione per evitare overfitting	78
11.7.1 Regolarizzazione ℓ_1 e ℓ_2	78
11.7.2 Dropout	79
11.7.3 Max-norm regularization	79
11.8 Tecniche per reti profonde	79
.1 FORMULE	80

Capitolo 1

Introduzione all'IA

Lezione 1 – 24/02/2026 – Russell & Norvig, Cap. 1 (1.1–1.5)

1.1 Che cos'è l'Intelligenza Artificiale?

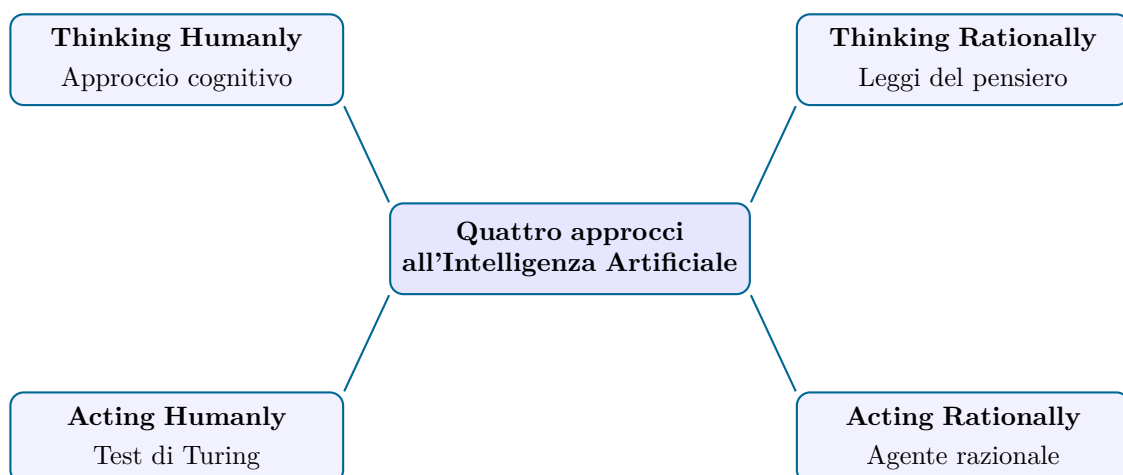
Definizione

L'**Intelligenza Artificiale** è la disciplina che studia la costruzione di *entità intelligenti*: macchine in grado di **calcolare come agire efficacemente e in modo sicuro** in un'ampia varietà di situazioni nuove.

L'IA è uno dei campi più interessanti e in rapida crescita di oggi, con un fatturato annuale superiore a **mille miliardi di dollari**. Abbraccia una vasta varietà di sottocampi: apprendimento, ragionamento, percezione, giochi, dimostrazione di teoremi, guida autonoma, diagnostica medica e molto altro.

Storicamente, la ricerca sull'IA ha seguito quattro grandi approcci, incrociando due dimensioni:

- **Umano vs. Razionale**: la macchina deve imitare le prestazioni umane o perseguire una razionalità astratta (fare la “cosa giusta”)?
- **Pensiero vs. Comportamento**: ci si concentra sui processi mentali interni o sul comportamento osservabile?



1.1.1 Approccio 1 – Acting Humanly: il Test di Turing

Alan Turing (1950) propose un esperimento mentale: un computer supera il test se un interrogatore umano, dopo aver posto domande scritte, non riesce a distinguere le risposte di una macchina da quelle di una persona.

Per superare il test una macchina deve possedere:

1. **Natural Language Processing (NLP)**: comunicare con successo in linguaggio umano.
2. **Knowledge Representation**: memorizzare ciò che percepisce o apprende.
3. **Automated Reasoning**: usare le informazioni salvate per rispondere e trarre conclusioni.
4. **Machine Learning**: adattarsi a nuove circostanze e rilevare pattern.

Il *Total Turing Test* aggiunge un segnale video, richiedendo anche **Computer Vision** e **Robotica**. Nonostante la sua eleganza, i ricercatori preferiscono studiare i principi sottostanti dell'intelligenza piuttosto che ottimizzare il superamento del test.

L'argomento della stanza cinese (Searle, 1980)

Superare il test non implica *capire*: un programma potrebbe manipolare simboli senza comprensione, come un uomo chiuso in una stanza che traduce ideogrammi cinesi seguendo manuali senza conoscere la lingua.

1.1.2 Approccio 2 – Thinking Humanly: la modellazione cognitiva

Per dire che un programma “pensa come un umano” bisogna prima capire come gli umani pensano. Tre metodi:

- **Introspezione**: osservare i propri pensieri mentre accadono.
- **Esperimenti psicologici**: osservare persone che svolgono compiti e confrontare il loro comportamento con programmi per computer.
- **Brain imaging**: osservare il cervello in azione (fMRI, EEG, MEG).

La **scienza cognitiva** combina modelli computazionali dell'IA con tecniche sperimentali della psicologia per costruire teorie precise e verificabili della mente umana. L'obiettivo è capire gli schemi mentali umani per riprodurli. Si basa su introspezione, esperimenti psicologici e brain imaging.

1.1.3 Approccio 3 – Thinking Rationally: le leggi del pensiero

Aristotele fu tra i primi a codificare il “pensiero corretto” tramite i **sillogismi** (“Socrate è un uomo; tutti gli uomini sono mortali; quindi Socrate è mortale”). Nel XIX secolo i logici svilupparono notazioni precise per ragionare su oggetti e relazioni.

- **Logica formale** (Boole 1847, Frege 1879): fondamento del ragionamento deduttivo in IA.
- **Teoria della probabilità** (Bayes, Laplace): estende la logica al ragionamento in condizioni di incertezza. (C'è il 30% di probabilità di pioggia)

Due ostacoli principali: è difficile tradurre la conoscenza informale in notazione formale e molti problemi, anche con pochi fatti, risultano intrattabili.

1.1.4 Approccio 4 – Acting Rationally: l'agente razionale

Agente razionale

Un **agente** è un'entità che agisce. Gli agenti software operano *autonomamente, percepiscono l'ambiente, persistono nel tempo, si adattano al cambiamento e perseguono obiettivi*.

Un **agente razionale** agisce in modo da ottenere il *miglior risultato atteso*, anche in condizioni di incertezza.

Questo approccio è **più generale** del “pensare razionalmente”: l'inferenza corretta è solo uno dei meccanismi per essere razionali. È anche più adatto allo sviluppo scientifico, perché la razionalità è definita matematicamente in modo preciso e universale. È l'approccio prevalente nell'IA moderna, nella teoria del controllo, nella ricerca operativa, nella statistica e nell'economia.

Il Modello Standard e i suoi limiti

Il “Modello Standard” dell'IA assume che il progettista conosca esattamente la funzione obiettivo. È pericoloso: se l'obiettivo è mal specificato si ottengono comportamenti indesiderati. Russell propone un modello alternativo: la macchina è incerta sull'obiettivo umano e impara a inferirlo.

1.2 Fondamenti dell'Intelligenza Artificiale

1.2.1 Filosofia

- **Dualismo** (Cartesio): la mente è separata dalla materia fisica ed è esente dalle leggi fisiche.
- **Empirismo** (Locke, Hume, Bacon): la conoscenza emerge dall'esperienza sensoriale. “Nulla è nell'intelletto che non sia stato prima nei sensi”.
- **Principio di induzione** (Hume): le regole generali si acquisiscono tramite l'esposizione ripetuta ad associazioni tra elementi.
- **Positivismo logico** (Circolo di Vienna): tutta la conoscenza è caratterizzata da teorie logiche collegate a osservazioni sensoriali.
- **Conoscenza e azione**: Aristotele propose che le azioni siano giustificate da connessioni logiche tra obiettivi e conoscenza degli esiti.

1.2.2 Matematica

- **Logica formale**: Boole (1847), logica proposizionale; Frege (1879), logica del primo ordine con oggetti e relazioni.
- **Probabilità**: Cardano, Pascal, Bayes, Laplace. La **regola di Bayes** è fondamentale per il ragionamento sotto incertezza in IA.
- **Calcolabilità**: Gödel (1931), Turing (1936), Church. La *Church–Turing thesis* identifica la calcolabilità con le macchine di Turing. Alcune funzioni non sono calcolabili.
- **Trattabilità**: la distinzione tra crescita polinomiale ed esponenziale è fondamentale. Molti problemi di IA sono NP-completi e richiedono un uso attento delle risorse computazionali.

1.2.3 Economia

- **Teoria delle decisioni:** combina probabilità e utilità per un framework formale per decisioni sotto incertezza (von Neumann & Morgenstern, 1944).
- **Teoria dei giochi:** analizza situazioni in cui le decisioni di più agenti si influenzano reciprocamente. Gli agenti razionali possono adottare strategie randomizzate.
- **Processi decisionali di Markov** (Bellman, 1957): formalizzano i problemi decisionali sequenziali, base del Reinforcement Learning.
- **Satisficing** (Simon): gli umani spesso prendono decisioni “abbastanza buone” piuttosto che ottimali – descrive meglio il comportamento reale.

1.2.4 Neuroscienze

Il **neurone** è l'unità base del sistema nervoso: corpo cellulare (soma), dendriti (input) e assone (output). Un neurone si connette con 10–100 000 altri neuroni alle sinapsi tramite reazioni elettrochimiche. Il cervello umano ha $\sim 10^{11}$ neuroni e $\sim 10^{14}$ sinapsi.

Cervello vs. Computer

Il cervello è 10^6 volte più lento nel ciclo di clock rispetto a un computer moderno, ma compensa con enorme capacità di storage e connettività.

1.2.5 Psicologia

- **Comportamentismo** (Watson, Skinner): rifiuta l'introspezione; studia solo le relazioni oggettive stimolo–risposta.
- **Psicologia cognitiva** (James, Craik): vede il cervello come un dispositivo di elaborazione dell'informazione. Le percezioni e le azioni sono mediate da rappresentazioni interne.
- **Modello di Craik (1943)**: (1) stimolo $\rightarrow$ rappresentazione interna; (2) manipolazione cognitiva; (3) retraduzione in azione.
- **Scienza cognitiva** (1956): combina modelli computazionali e psicologia sperimentale; Miller, Chomsky, Newell & Simon dimostrarono che i computer possono modellare memoria, linguaggio e ragionamento.

1.2.6 Ingegneria informatica

- I moderni calcolatori nascono durante la Seconda Guerra Mondiale: **Colossus** (UK, 1943), **Z-3** (Zuse, 1941), **ENIAC** (USA, 1945).
- **Legge di Moore**: la potenza computazionale raddoppia ogni 18 mesi, grazie a questa legge, le prestazioni dei PC sono passate dai megaFLOP (milioni di operazioni in virgola mobile al secondo) agli attuali exaFLOP (miliardi di miliardi di operazioni al secondo).
- **Hardware specializzato**: GPU (NVIDIA), TPU (Google), WSE per l'addestramento di modelli di ML. Dal 2012 al 2018, la potenza per il training è cresciuta di $300.000\times$.
- **Quantum computing**: Il bit classico vale 0 o 1. Il Qubit (bit quantistico), grazie al principio di sovrapposizione, può assumere infiniti valori compresi tra 0 e 1 contemporaneamente.

1.2.7 Teoria del controllo e cibernetica

- Sistemi di feedback autoregolanti precedono i computer: orologio ad acqua (Ktesibios), regolatore del motore a vapore (Watt), termostato (Drebbel).
- **Teoria del controllo** (Maxwell, Wiener): framework matematico per sistemi che minimizzano l'errore tra stato corrente e stato obiettivo.
- **Cibernetica** (Wiener, 1948): studia il comportamento finalistico che emerge da meccanismi regolatori in macchine e organismi.
- **Controllo ottimo stocastico**: progetta sistemi che massimizzano una funzione obiettivo nel tempo – parallelo diretto con l'obiettivo dell'IA.

1.2.8 Linguistica

- **Comportamentismo linguistico** (Skinner, 1957): il linguaggio si apprende per associazioni stimolo-risposta.
- **Strutture sintattiche** (Chomsky, 1957): il linguaggio mostra creatività – i bambini comprendono e producono frasi mai sentite prima. La sintassi formale di Chomsky spiegò questo fenomeno e poteva essere programmata.
- Linguistica moderna e IA nacquero quasi contemporaneamente, dando origine alla **linguistica computazionale** e al **Natural Language Processing**.
- Insight chiave: comprendere il linguaggio richiede conoscenza del dominio e del contesto, non solo regole sintattiche – legato alla **rappresentazione della conoscenza**.

1.3 Storia dell'Intelligenza Artificiale

1.3.1 Le origini (1943–1956)

- **McCulloch & Pitts (1943)**: primo modello formale del neurone artificiale; qualsiasi funzione calcolabile può essere computata da reti di neuroni.
- **Hebb (1949)**: regola di apprendimento hebbiano – “neurons that fire together wire together”.
- **Turing (1950)**: propone il Turing Test, introduce machine learning e algoritmi genetici.
- **Conferenza di Dartmouth (1956)**: McCarthy, Minsky, Shannon e Rochester organizzano il workshop – **nascita ufficiale dell'IA** come campo.

1.3.2 Entusiasmo e aspettative (1952–1969)

- **Logic Theorist** (Newell & Simon, 1956): dimostra teoremi di logica formale; Russell rimase deliziato quando il programma trovò una dimostrazione più breve della sua.
- **General Problem Solver** (anni '60): imita i protocolli umani di problem solving – primo approccio “Thinking Humanly”.
- **LISP** (McCarthy, 1958): linguaggio dominante dell'IA per 30 anni.
- **Microworlds** (anni '60): domini limitati (es. il blocchi-mondo di SHRDLU) dove il problem solving sembrava richiedere intelligenza.

1.3.3 Una dose di realtà e i primi inverni (1966–1986)

- I sistemi fallivano su problemi più grandi per **esplosione combinatoria**: scalare a istanze reali si rivelò impraticabile.
- **Minsky & Papert (1969)**: dimostrano i limiti del perceptrone (non può risolvere XOR) → blocca la ricerca sulle reti neurali per anni.
- **Rapporto Lighthill (1973)**: critica i fallimenti dell'IA → taglio dei finanziamenti britannici.
- **Sistemi esperti** (DENDRAL 1969, MYCIN anni '70, R1/XCON 1982): primo successo commerciale – conoscenza di dominio specifica invece di metodi deboli generali. Ma fragili di fronte a incertezza e casi nuovi → secondo inverno (fine anni '80).

1.3.4 Rinascita e Deep Learning (1986–oggi)

- **Backpropagation** (Rumelhart, Hinton, Williams 1986): ri-scoperta e applicazione a molti problemi di apprendimento.
- **Reti bayesiane** (Pearl, 1988): framework rigoroso per il ragionamento probabilistico.
- **Big Data** (2001–oggi): raddoppiare i dati supera qualsiasi miglioramento algoritmico (Banko & Brill, 2001).
- **ImageNet 2012**: AlexNet di Hinton segna l'inizio del dominio del **Deep Learning**.
- Pietre miliari recenti: **AlphaGo** (2016), **Transformer** (2017), **GPT-3** e **AlphaFold** (2020), **ChatGPT** (2022), multimodalità e agenti autonomi (2024+).

1.4 Stato dell'arte

Le pubblicazioni di IA sono aumentate di 20 volte tra il 2010 e il 2019.

Applicazioni reali oggi

- **Sanità**: diagnosi di malattie, scoperta di farmaci.
 - **Guida autonoma**: veicoli STANLEY (DARPA 2005), sistemi Waymo, Tesla.
 - **NLP**: traduzione automatica, assistenti vocali, LLM (GPT-4, Gemini, Claude).
 - **Scienza**: AlphaFold (DeepMind, 2020) ha risolto la predizione 3D delle proteine.
 - **Giochi**: Deep Blue (scacchi, 1997), AlphaGo (Go, 2016), AlphaStar (2019).
 - **Spam filtering**: algoritmi di ML classificano un miliardo di messaggi al giorno.
-
- **Interpretabilità (xAI)**: i sistemi DL sono spesso *black box*. LIME, SHAP, Grad-CAM tentano di renderli trasparenti.
 - **Robustezza**: vulnerabilità a *adversarial examples* e input fuori distribuzione.
 - **Generalizzazione**: trasferire la conoscenza da un dominio all'altro rimane difficile.
 - **Fairness e bias**: i sistemi di IA possono amplificare i pregiudizi nei dati di addestramento.
 - **Consumo energetico**: il training di modelli grandi consuma enormi quantità di energia (GPT-3: ~1287 MWh).

1.5 Rischi e benefici dell'IA

1.5.1 Il Problema dell'Allineamento dei Valori

Modello Standard

Il **Modello Standard** dell'IA prevede la progettazione di sistemi che raggiungono un obiettivo specificato dall'utente. Assume che possiamo specificare completamente l'obiettivo. In realtà, in problemi aperti del mondo reale, questo è spesso impossibile.

Value Alignment Problem

Il **problema dell'allineamento dei valori** è la sfida di garantire che gli obiettivi programmati in una macchina siano *allineati con i valori umani*. Un sistema sufficientemente capace che persegue un obiettivo fisso potrebbe trovare modi inattesi per raggiungerlo.

Verso un'IA provabilmente benefica (Russell, *Human Compatible*, 2019)

Poiché non possiamo trasferire tutti gli obiettivi a una macchina, serve un approccio in cui la macchina persegue i nostri obiettivi *rimanendo incerta su quali siano davvero*. Questo la incentiva a: agire con **cautela**, chiedere **permesso** prima di azioni importanti, **apprendere** le preferenze umane, e **delegare il controllo** all'uomo in caso di incertezza.

1.5.2 Rischi e benefici

Principali benefici

- **Sanità:** diagnosi, scoperta di farmaci, medicina personalizzata.
- **Scienza:** accelera la ricerca in fisica, biologia, chimica.
- **Economia:** automazione, ottimizzazione, supporto decisionale.
- **Accessibilità:** strumenti per persone con disabilità, risposta alle emergenze.

Principali rischi

- **Disoccupazione tecnologica:** l'automazione può eliminare posti di lavoro più rapidamente di quanto ne crei.
- **Bias e discriminazione:** i sistemi amplificano i pregiudizi nei dati.
- **Disinformazione:** deepfake, testo sintetico e media manipolati diffondono informazioni false su scala.
- **Privacy e sorveglianza:** la raccolta massiva di dati abilita una sorveglianza senza precedenti.
- **Concentrazione del potere:** i benefici dell'IA rischiano di concentrarsi in poche grandi aziende o nazioni.
- **Armi autonome:** sistemi militari basati sull'IA sollevano gravi preoccupazioni etiche ed esistenziali.

- **AI Act UE (2024):** prima regolamentazione orizzontale sull'IA; impone requisiti di trasparenza, robustezza e supervisione umana in base al livello di rischio.
- **Collaborazione interdisciplinare:** informatici, etici, filosofi, scienziati sociali ed esperti di dominio devono lavorare insieme.
- **Ricerca fondamentale:** progressi su interpretabilità, robustezza, fairness e allineamento rimangono essenziali.

Capitolo 2

Agenti Intelligenti

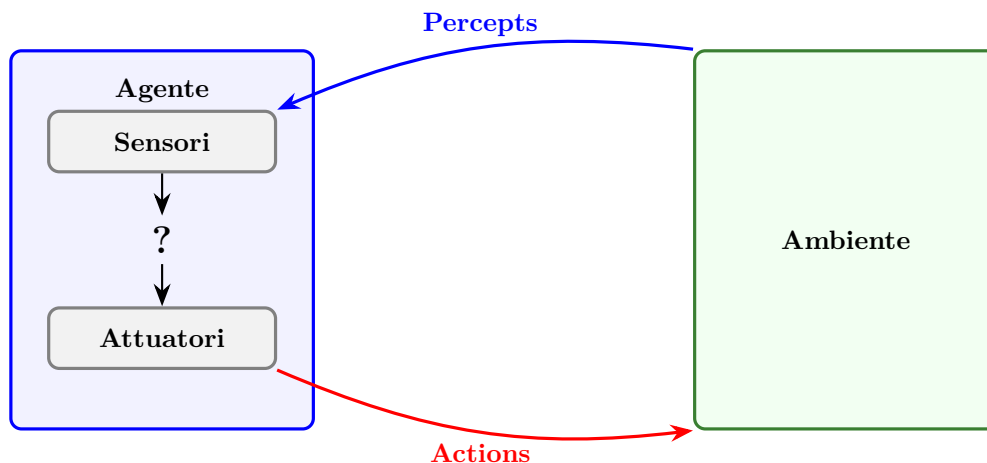
Lezione 2 – 03/03/2026 – Russell & Norvig, Cap. 2 (§2.1–2.4)

L'idea di **agente razionale** è il filo conduttore di tutto il libro di Russell & Norvig: invece di chiederci se una macchina “pensa”, ci chiediamo se *agisce nel modo migliore possibile* per raggiungere i propri obiettivi, data l'evidenza disponibile.

2.1 Agenti e interazione con l'ambiente

Agente

Un **agente** è qualsiasi entità che *percepisce* il proprio ambiente tramite **sensori** e *agisce* su di esso tramite **attuatori**.



Esempi di agenti:

- **Agente umano:** sensori = vista, udito, tatto, olfatto; attuatori = mani, piedi, voce.
- **Agente robotico:** sensori = telecamere, infrarossi, sonar, accelerometri; attuatori = motori, pistoni, altoparlanti.
- **Agente software (*softbot*):** riceve input da file, rete, tastiera, API; agisce scrivendo file, inviando pacchetti di rete, visualizzando informazioni.

Esempi pratici

Agricoltura intelligente: un sensore di acidità del terreno percepisce i dati per regolare fertilizzanti e irrigazione (attuatori).

Domotica: un sensore di illuminazione naturale attiva l'abbassamento/alzamento delle tapparelle o l'accensione del riscaldamento.

2.1.1 Percetti, sequenza percettiva e funzione agente

- **Percetto:** il contenuto percepito dai sensori in un dato istante.
- **Sequenza percettiva:** la storia completa di tutto ciò che l'agente ha mai percepito.

La scelta dell'azione può dipendere dalla conoscenza pregressa e dall'intera sequenza percettiva, ma *non* da ciò che l'agente non ha percepito.

Funzione agente e Programma agente

La **funzione agente** $f : \mathcal{P}^* \rightarrow \mathcal{A}$ è la descrizione matematica (astratta) che mappa ogni possibile sequenza percettiva in un'azione.

Il **programma agente** è la sua implementazione concreta, eseguita su un'**architettura** fisica (hardware con sensori e attuatori).

$$\text{Agente} = \text{Architettura} + \text{Programma}$$

2.1.2 Il mondo dell'aspirapolvere**Esempio canonico: il mondo dell'aspirapolvere (Russell & Norvig)**

Due caselle (A e B), ciascuna pulita o sporca. Percetti: posizione + stato della casella. Azioni: **Aspira**, **Sinistra**, **Destra**.

Funzione agente semplice (agente reflex):

1. Se la casella corrente è sporca $\rightarrow$ **Aspira**.
2. Se la casella è pulita e l'agente è in $A \rightarrow$ **Destra**.
3. Se la casella è pulita e l'agente è in $B \rightarrow$ **Sinistra**.

Misura di prestazione corretta: un punto per ogni casella pulita ad ogni passo temporale, con penalità per spostamenti ed energia consumata – *non* “quantità di sporco rimossa” (altrimenti l'agente potrebbe spargere sporco per ripulirlo!).

Algoritmo del serpente

In ambienti più grandi (es. griglia 10×10), un agente reflex di base può implementare l'**algoritmo del serpente**: percorre ogni riga da sinistra a destra, poi scende di una riga e torna da destra a sinistra, e così via. È il metodo usato nei sistemi automatici di raccolta ordini nei magazzini. Non è però ottimale in ambienti dinamici dove la sporcizia riappare.

2.2 Razionalità e misure di prestazione

Agente razionale

Per ogni possibile sequenza percettiva, un **agente razionale** deve selezionare l'azione che ci si aspetta *massimizzi* la sua **misura di prestazione**, sulla base dell'evidenza fornita dalla sequenza percettiva e dalla propria conoscenza pregressa.

È meglio progettare le misure di prestazione in base a *ciò che si vuole davvero ottenere nell'ambiente*, non in base a come si pensa che l'agente dovrebbe comportarsi.

2.2.1 I 4 pilastri della razionalità

La razionalità dipende da quattro fattori concomitanti:

1. **La misura di prestazione**: il criterio di successo (es. pulizia media nel tempo; valutazione dell'aspirapolvere in base alla pulizia raggiunta rispetto alla sporcizia totale; la qualità è data anche dalla quantità di spostamenti necessari per pulire l'ambiente).
2. **La conoscenza pregressa**: ciò che l'agente conosce già dell'ambiente prima di agire.
3. **Le azioni eseguibili**: il repertorio di operazioni permesse dagli attuatori.
4. **La sequenza percettiva**: la storia completa di tutto ciò che l'agente ha percepito.

2.2.2 Razionalità vs. onniscienza e autonomia

La razionalità **non** è perfezione né onniscienza.

- Un agente **onnisciente** conosce l'esito reale delle azioni – ma l'onniscienza è impossibile nella realtà.
- La **razionalità** massimizza la prestazione *attesa*; la **perfezione** massimizza la prestazione *effettiva*.

Un agente dovrebbe scegliere un'azione che fornisca le migliori prestazioni sulla base del percepito e della conoscenza che ha (es. sa quali sono le dimensioni del mondo in cui si trova?). La scelta razionale di una persona può sembrare irrazionale a un'altra, se questa non è a conoscenza del suo punto di vista: *perché hai fatto così o non colà?*.

Non basta raccogliere le informazioni, bisogna percepirle. Qui subentrano i **limiti della razionalità**:

- **Filosofia e libero arbitrio**: La razionalità ha un limite che richiama interpretazioni filosofiche e teologiche sul libero arbitrio. L'uomo può scegliere, quindi è autonomo. Se non avesse possibilità di scelta, sarebbe un burattino nelle mani dell'onnisciente nella ricerca del bene.

Scelta vincolata vs. libera

Il treno si muove vincolato sulle rotaie (nessuna scelta), l'auto si può muovere su diversi percorsi scelti.

- **Conoscenza incompleta:** L'agente non ha sempre la completa conoscenza dell'ambiente.

Auto a guida autonoma

L'auto a guida autonoma e il bimbo che attraversa la strada all'improvviso. In questi casi, l'autonomia di giudizio completa la parziale conoscenza.

- **Onniscienza limitata:** Un agente onnisciente conosce l'esito delle sue azioni e sceglie quella per il miglior risultato.

Giochi a turni

Nel gioco del tris o degli scacchi, l'onniscienza è per forza parziale/limitata perché non si ha la certezza della mossa successiva dell'avversario.

Razionale ma sfortunato (R&N)

Attraversi la strada dopo aver guardato in entrambe le direzioni. Un carico cade da un aereo di passaggio e ti colpisce. Hai agito *razionalmente* (la tua scelta era ottima data l'evidenza disponibile), ma il risultato è stato pessimo. La razionalità si giudica *ex ante*, non *ex post*.

Un agente razionale deve anche:

- **Raccogliere informazioni** (*information gathering*): guardare prima di attraversare, esplorazione di ambienti ignoti.
- **Apprendere** dall'esperienza per compensare conoscenza pregressa parziale o errata.
- Essere **autonomo**: il comportamento deve essere determinato dalla propria esperienza, non solo dalla conoscenza pre-programmata dal progettista.

Autonomia

Un agente che si affida solo alla conoscenza pre-programmata *manca di autonomia*. Così come l'evoluzione fornisce agli animali riflessi innati per sopravvivere abbastanza a lungo da imparare, un agente artificiale può partire con una conoscenza iniziale ma deve acquisire autonomia attraverso l'apprendimento.

2.3 La natura degli ambienti: PEAS

PEAS – Task Environment

Per progettare un agente, il primo passo è specificare il **task environment** nel modo più completo possibile. Si usa l'acronimo **PEAS**:

- **Performance measure** – criterio di successo
- **Environment** – l'ambiente esterno
- **Actuators** – gli attuatori disponibili
- **Sensors** – i sensori disponibili

PEAS per un taxi autonomo

- **P:** viaggio sicuro, veloce, legale, confortevole; massimizzare i profitti.
- **E:** strade, altri veicoli, pedoni, segnali stradali, meteo, clienti.
- **A:** sterzo, acceleratore, freni, frecce, clacson, display di bordo.
- **S:** videocamere, lidar, sonar, GPS, tachimetro, accelerometri, microfono.

2.3.1 Dimensioni di classificazione degli ambienti

Dimensione	Opzioni	Descrizione ed esempi
Osservabilità	Tot. osservabile	I sensori dell'agente gli consentono di accedere allo stato completo dell'ambiente in ogni momento (es. sensore di temperatura, scacchi)
	Parz. osservabile	I sensori dell'agente sono rumorosi o imprecisi (es. guida di un taxi)
	Inosservabile	L'agente non ha sensori
Numero di agenti	Singolo	Agente unico (es. cruciverba)
	Multi-agente	Più agenti interagiscono e i loro comportamenti si influenzano a vicenda (es. auto a guida autonoma)
Determinismo	Deterministico	Stato successivo certo (es. l'aspirapolvere pulisce la cella sporca)
	Stocastico	Subentra la probabilità (es. sacchetto dell'aspirapolvere pieno)
Episodicità	Episodico	L'azione è divisa in episodi atomici e indipendenti (es. passi di una ricetta)
	Sequenziale	Le azioni sono legate (es. parole crociate)
Dinamicità	Dinamico	L'ambiente evolve nel tempo (es. cesta dei panni sporchi)
	Statico	L'ambiente non cambia mentre l'agente delibera (es. esame superato)
	Semi-dinamico	L'ambiente non cambia ma la prestazione nel tempo sì (es. scacchi)
Discrezione	Discreto	Stati ben definiti (es. tiro di un dado, numeri al lotto)
	Continuo	Variabili continue nello spazio e nel tempo (es. guida di un taxi)
Notorietà	Noto	Esiti conosciuti (es. lancio di un dado)
	Ignoto	Esiti sconosciuti (es. poker con bluff ma esito ignoto)

L'ambiente più difficile

L'ambiente più complesso è: *parzialmente osservabile, multi-agente, stocastico, sequenziale, dinamico, continuo, ignoto* – il mondo reale. La guida del taxi è difficile su tutte queste dimensioni tranne che la conoscenza dell'ambiente (le leggi della fisica e del traffico sono note).

La tabella seguente riassume la classificazione di alcuni ambienti tipici:

Task environment	Osservabile	Agenti	Determinismo	Episodico	Dinamistico	Discreto
Cruciverba	Totalmente	Singolo	Deterministico	Sequenziale	Statico	Discreto
Scacchi (con orologio)	Totalmente	Multi	Deterministico	Sequenziale	Semi-dinamico	Discreto
Poker	Parzialmente	Multi	Stocastico	Sequenziale	Statico	Discreto
Guida del taxi	Parzialmente	Multi	Stocastico	Sequenziale	Dinamico	Continuo
Diagnosi medica	Parzialmente	Singolo	Stocastico	Sequenziale	Dinamico	Continuo
Analisi immagini	Totalmente	Singolo	Deterministico	Episodico	Semi-dinamico	Continuo

2.4 Struttura e tipologie degli agenti

Il **programma agente** implementa la funzione agente (la mappatura percetti $\rightarrow$ azioni). Esistono quattro architetture di base, in ordine crescente di complessità.

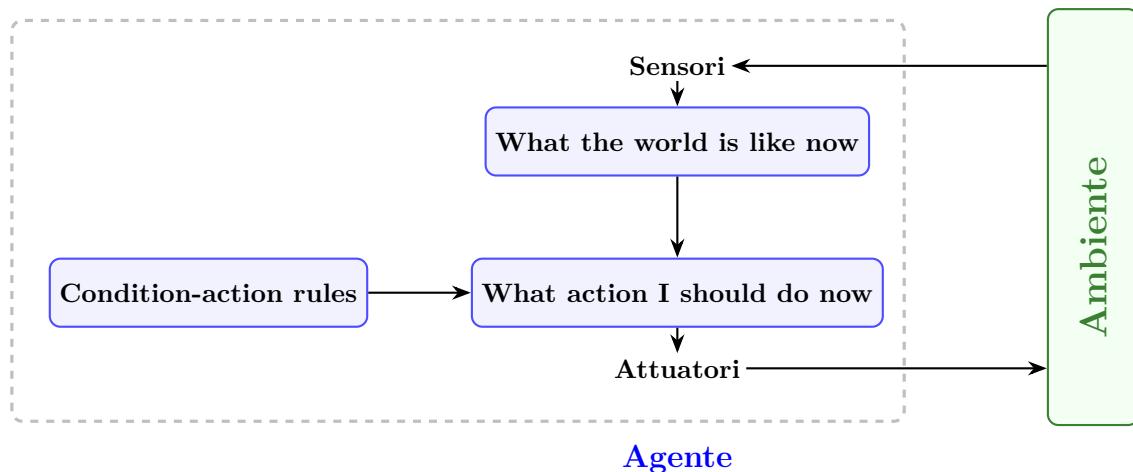
2.4.1 1. Agenti Reflex Semplici

Sono la tipologia più basilare di agente intelligente: selezionano le azioni basandosi esclusivamente sulla **percezione corrente**, ignorando completamente lo storico delle percezioni passate.

Si basano su **regole condizione-azione** (*condition-action rules*) espresse nella forma **if** [condizione] **then** [azione]. Il processo avviene in 3 fasi:

1. **Percezione:** i sensori catturano i dati dall'ambiente.
2. **Elaborazione:** l'agente converte la percezione in una descrizione della situazione corrente.
3. **Corrispondenza:** quando la percezione soddisfa la "condizione" di una regola, viene attivata l'"azione" associata.

L'agente reflex è razionale *solo* se l'ambiente è completamente osservabile. Il limite principale è la **parziale osservabilità**, che può provocare un **loop infinito** in cui l'agente oscilla tra gli ambienti e spreca energia. Un esempio è il robot aspirapolvere base che agisce solo alla presenza di sporco nella cella in cui si trova in quel preciso istante, senza ricordare le stanze già pulite.



Il flusso decisionale dell'agente reflex:

1. I sensori ricevono una percezione dell'ambiente in un dato istante.
2. L'agente interpreta l'input sensoriale per definire la situazione corrente.
3. Applicazione delle regole condizione-azione.
4. Viene selezionata l'azione corrispondente alla percezione attuale.
5. L'azione viene inviata agli attuatori per essere eseguita.

Pseudocodice agente reflex aspirapolvere

```

function REFLEX-VACUUM-AGENT([location, status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left

```

- **Vantaggi:** estrema semplicità, basso costo computazionale, efficienza computazionale, semplicità di implementazione, basso consumo di risorse.
- **Limiti:** funzionano *solo* in ambienti completamente osservabili. In ambienti parzialmente osservabili, l'agente può cadere in **loop infiniti** (es. aspirapolvere senza sensore di posizione che oscilla tra *A* e *B*). Inoltre, manca di **adattabilità**: non può imparare dai propri errori e fallisce in ambienti dinamici.
- **Soluzione parziale:** lo spostamento casuale (*random*) dell'agente in un altro ambiente permette di uscire dal loop senza dover memorizzare lo stato passato, ma non è ottimale.
- **Nota:** l'agente reflex non mantiene alcuna memoria delle percezioni passate, quindi ogni decisione è basata esclusivamente sull'input sensoriale corrente.
- **Applicazioni tipiche:** termostati, rilevatori di fumo, sistemi di controllo semplici dove l'ambiente è completamente osservabile e le regole sono ben definite.

2.4.2 2. Agenti Reflex Basati su Modello

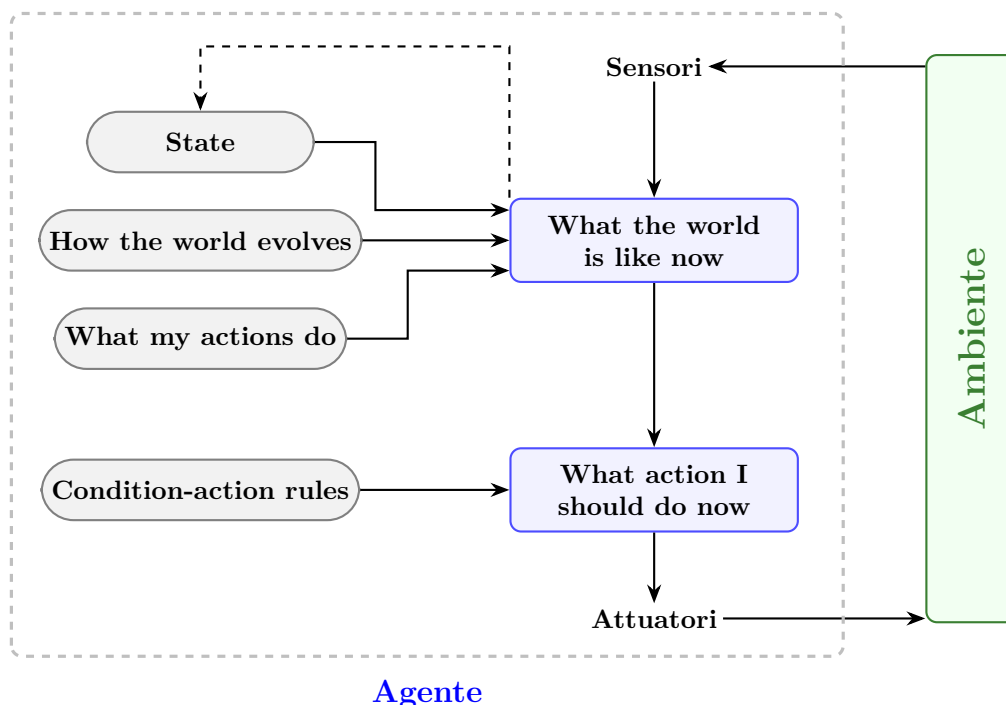
L'agente **model-based** è stato progettato per gestire la **parziale osservabilità** dell'ambiente mantenendo uno **stato interno** che tiene traccia della parte di mondo che l'agente non può vedere in quel momento. Questo stato dipende dalla storia delle percezioni e riflette gli aspetti inosservabili dello stato attuale.

L'agente conosce:

- Lo **stato** che rappresenta la memoria dell'agente.
- **Modello di transizione**: la conoscenza di come il mondo evolve nel tempo in modo indipendente dall'agente, e di come le azioni dell'agente stesso modifichino l'ambiente (effetti delle azioni + evoluzione autonoma). Descrive come cambia l'ambiente e gli effetti delle azioni dell'agente sul mondo.
- **Modello sensoriale**: come lo stato del mondo si riflette nelle percezioni dei sensori. Descrive come lo stato del mondo si rifletta nelle percezioni dell'agente.

Robot aspirapolvere model-based

Il robot ricorda che dietro di sé c'è un muro, anche se i sensori frontali non lo vedono in questo momento. Lo stato interno conserva la mappa parziale dell'ambiente esplorato.



Il flusso decisionale dell'agente model-based:

1. **Percezione**: i sensori ricevono dati dall'ambiente.
2. **Aggiornamento dello stato**: l'agente determina com'è il mondo ora per colmare le parti non visibili, combinando lo stato precedente, l'ultima azione compiuta, la percezione attuale e i modelli di transizione e sensoriale.
3. **Applicazione delle regole** condizione-azione.
4. **Decisione**: scelta dell'azione che l'agente deve eseguire cercando una regola corrispondente allo stato attuale.
5. **Esecuzione**: l'azione viene inviata agli attuatori per influenzare l'ambiente.

Pseudocodice agente model-based

```

function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
               model, a description of how the next state depends
                   on current state and action
               rules, a set of condition-action rules
               action, the most recent action, initially none

  state UPDATE-STATE(state, action, percept, model)
  rule  RULE-MATCH(state, rules)
  action rule.ACTION
  return action

```

2.4.3 3. Agenti Basati sull'Obiettivo

Oltre allo stato corrente, dispongono di **informazioni sull'obiettivo** (*goal*): descrivono situazioni desiderabili (es. trovarsi alla destinazione del passeggero). Il programma combina il modello del mondo con il goal per scegliere azioni che lo raggiungano. Il vantaggio è la **flessibilità**: contrariamente agli agenti riflessi che hanno regole fisse, se l'obiettivo cambia, il comportamento dell'agente si adatta automaticamente per raggiungere il nuovo obiettivo. Un esempio è il robot aspirapolvere il cui obiettivo è pulire tutta la scacchiera/casa e tornare alla base.

- Quando la selezione non è banale, si usano algoritmi di **ricerca e pianificazione**.
- **Vantaggio**: la conoscenza è rappresentata *esplicitamente* e può essere modificata. Per cambiare destinazione basta specificare un nuovo goal – un agente reflex dovrebbe riscrivere tutte le sue regole.

2.4.4 4. Agenti Basati sull'Utilità

I goal forniscono solo una distinzione binaria (“felice” o “infelice”). Spesso molte sequenze di azioni raggiungono l'obiettivo, ma alcune sono più rapide, sicure o economiche. Serve quindi uno strumento più preciso: la **funzione di utilità**.

Quando ci sono obiettivi in conflitto o incertezza sul successo di un'azione, l'agente non può limitarsi a chiedersi “raggiungo o non raggiungo il goal?”. Deve invece valutare quanto sia *buono* un determinato stato. La funzione di utilità assegna a ogni stato un valore numerico che rappresenta il “grado di soddisfazione” dell'agente in quello stato.

Un agente razionale basato sull'utilità sceglie l'azione che massimizza l'**utilità attesa**, cioè la media dei valori degli stati possibili, pesata per la probabilità che ciascuno si verifichi. In questo modo può gestire:

- **Obiettivi in conflitto**: ad esempio, raggiungere la destinazione nel minor tempo possibile ma anche consumare poca benzina.
- **Incertezza**: se un'azione può portare a esiti diversi con certe probabilità, l'agente sceglie quella con il miglior risultato atteso.

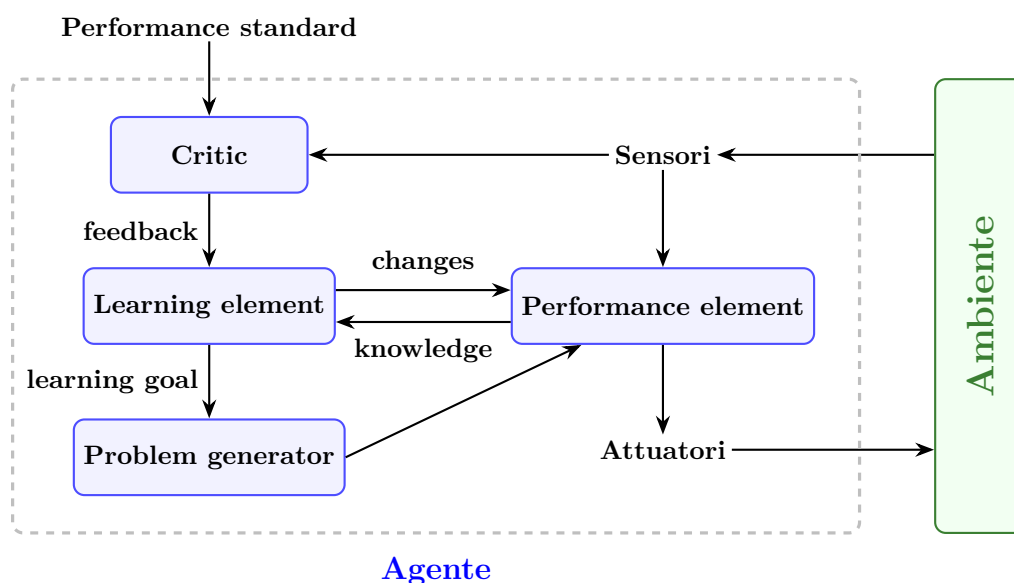
Auto a guida autonoma

L'obiettivo è arrivare a destinazione, ma la funzione di utilità combina: sicurezza (peso elevato), tempo di percorrenza, consumo di carburante, rispetto del codice della strada, comfort dei passeggeri. L'agente sceglie velocità e traiettoria che massimizzano l'utilità attesa.

2.4.5 5. Agenti che Apprendono

Qualsiasi tipo di agente (reflex, model-based, goal-based, utility-based) può essere dotato di **capacità di apprendimento**. L'apprendimento permette all'agente di operare in ambienti inizialmente ignoti e di migliorare le proprie prestazioni nel tempo.

Un **learning agent** è strutturato in quattro componenti:



- **Performance element:** l'agente vero e proprio – percepisce e agisce in base a ciò che conosce.
- **Critic:** valuta le prestazioni dell'agente rispetto a uno standard esterno fisso; passa il feedback al learning element.
- **Learning element:** responsabile dei miglioramenti – modifica la conoscenza del performance element.
- **Problem generator:** suggerisce azioni esplorative che potrebbero portare a esperienze informative (anche subottimali nel breve termine, migliori nel lungo).

Taxi che impara

Il performance element guida il taxi. Dopo una svolta brusca, il critic osserva la reazione degli altri guidatori (linguaggio colorito, frenate brusche) e passa il segnale negativo al learning element, che aggiunge la regola "svolta brusca = cattiva azione". Il problem generator potrebbe suggerire di testare i freni su asfalto bagnato per imparare la distanza di frenata.

2.4.6 Rappresentazioni dello stato

I componenti dell'agente possono rappresentare l'ambiente con livelli crescenti di espressività:

1. **Atomica**: ogni stato è indivisibile, senza struttura interna (es. nome di una città in un problema di percorso).
2. **Fattorizzata**: ogni stato è un vettore di variabili/attributi con valori (es. reti bayesiane, logica proposizionale).
3. **Strutturata**: oggetti con attributi e relazioni esplicite tra loro (es. logica del primo ordine, database relazionali).

Rappresentazioni **localiste** (un concetto = una locazione di memoria) vs. **distribuite** (un concetto = pattern su molte unità, tipiche delle reti neurali artificiali – più robuste al rumore).

Capitolo 3

Introduzione al Machine Learning

Lezione 3 – 09/03/2026 – Géron, Cap. 1

3.1 Cos'è il Machine Learning?

Definizione informale (Arthur Samuel, 1959)

Il *Machine Learning* è “il campo di studio che dà ai computer la capacità di imparare **senza essere esplicitamente programmati**”.

Definizione formale (Tom Mitchell, 1997)

Un programma impara dall'esperienza E rispetto a un task T , misurando le proprie performance tramite un indice P , se le sue prestazioni nel task T , misurate con P , *migliorano* con l'esperienza E .

Il Machine Learning è al tempo stesso *scienza* (matematica, statistica, ottimizzazione) e *arte* (sensibilità sui dati, scelta delle feature, intuizione del modello giusto). Consiste nel programmare i computer in modo che **apprendano dai dati**, senza essere programmati esplicitamente per ogni singola eccezione.

Applicazioni quotidiane del ML

Filtri antispam, OCR, traduzione automatica (Google Translate, DeepL), ricerca per immagini, assistenti vocali (Siri, Alexa), sistemi di raccomandazione (Netflix, Amazon, Spotify), rilevazione frodi con carte di credito, diagnosi medica da immagini, autocompletamento del testo, navigazione GPS adattiva. L'obiettivo dei social network è creare sistemi che etichettano e profilano gli utenti per scopi di marketing – un classificatore ML decide quali contenuti mostrare a ciascun utente.

3.2 Come funziona l'apprendimento

Pensiamo a una **classificazione binaria**: distinguere fotografie di gatti da fotografie di non-gatti. Questo tipo di apprendimento imita l'età infantile: i bambini capiscono cos'è un oggetto tramite l'esperienza, non tramite definizioni formali.

Il programma *non può* basarsi su regole fisse: “se ha 4 zampe è un gatto” confonderebbe il cane; “se ha la coda” confonderebbe pappagalli e pesci. Ha bisogno di:

1. **Esperienza** (E): dati *etichettati* (immagini con label “gatto” / “non gatto”).
2. **Task** (T): classificare nuove immagini correttamente.
3. **Misura di performance** (P): es. accuratezza – percentuale di immagini classificate correttamente.

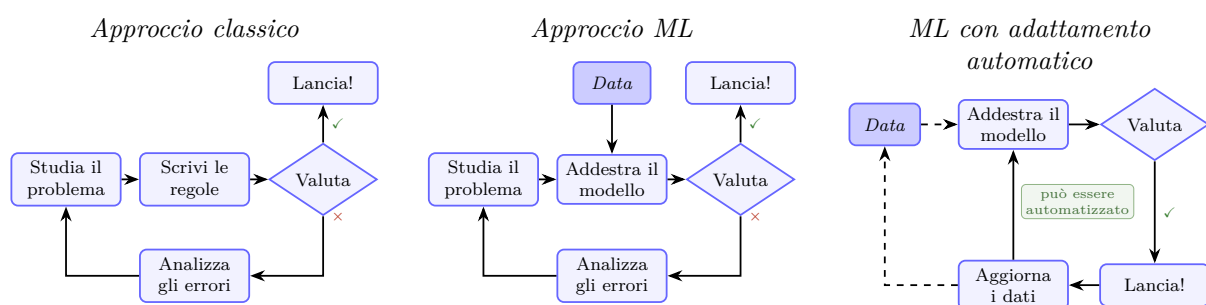
Formula fondamentale:

$$\text{Modello} = \text{Algoritmo} + \text{Dati dell'esperienza}$$

3.3 Approccio classico vs. Machine Learning

L'esempio storico è il **filtro antispam**. Inizialmente usava condizioni **if-else**.

	Approccio classico	Machine Learning
Metodo	Regole if-else scritte a mano	Addestramento su dati etichettati
Aggiornamento	Riscrivere le regole manualmente	Riaddestrare il modello su nuovi dati
Adattamento	Nessuno, statico	Automatico al cambiare dei dati
Punto di forza	Semplice per problemi piccoli e stabili	Flessibile, scopre pattern nascosti
Tipico esempio	Calcolo IVA	Rilevazione spam, OCR, traduzione



Se lo spammer cambia tattica e scrive “For U” invece di “4U”, il filtro classico deve essere aggiornato a mano. Un filtro ML nota automaticamente che “For U” è diventato frequente nelle email segnalate come spam e inizia a bloccarle – senza intervento umano.

Il ML **eccelle** quando:

- Le regole sono troppe o cambiano frequentemente (lo spam evolve continuamente).
- Il problema è troppo complesso per regole esplicite (riconoscimento del parlato in lingue diverse, in ambienti rumorosi, con accenti variabili).
- Si vogliono scoprire pattern nascosti in grandi dataset (*data mining*).
- L'ambiente cambia e il modello deve adattarsi (*online learning*).

3.4 Training set e Test set

- **Training set** (~80%): l'insieme di dati usato per *addestrare* il modello e accumulare esperienza.
- **Test set** (~20%): l'insieme di dati usato per *valutare* le prestazioni su dati mai visti, prima del rilascio in produzione.

Regola d'oro – Data Snooping Bias

Il test set *non deve mai essere usato durante lo sviluppo*. Se si consulta il test set per prendere decisioni sul modello, si commette **data snooping bias**: le prestazioni misurate saranno ottimistiche e non rappresentative delle prestazioni reali su nuovi dati.

3.5 La Matrice di Confusione

Per valutare un classificatore binario si usa la **matrice di confusione**, che mappa i quattro possibili esiti:

		Valori Reali	
		Positivo (1)	Negativo (0)
Valori Predetti	Pos. (1)	TP Vero Positivo	FP Falso Positivo
	Neg. (0)	FN Falso Negativo	TN Vero Negativo

Il classificatore ideale restituisce solo **TP** e **TN**; la presenza di **FP** e **FN** indica gli errori commessi.

Interpretazione pratica

Nel classificatore “gatto / non gatto”:

- **TP**: il modello dice “gatto” ed è un gatto.
- **TN**: il modello dice “non gatto” ed è un non gatto.
- **FP**: il modello dice “gatto” ma è un cane (*falso allarme*).
- **FN**: il modello dice “non gatto” ma è un gatto (*mancato rilevamento*).

In un sistema di diagnosi medica, un **FN** (tumore non rilevato) è molto più pericoloso di un **FP** (paziente sano rimandato per ulteriori esami). La scelta della soglia di classificazione bilancia questi due tipi di errore.

Il classificatore ideale restituisce solo **TP** e **TN**. Le metriche derivate sono:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

3.6 Struttura di un Dataset

Un dataset è una matrice strutturata in:

- **Istanze** (righe): i singoli casi osservati (es. un'email, un'immagine, un paziente).
- **Feature** (colonne): gli attributi o caratteristiche di ogni istanza (es. frequenza di parole, pixel, valori clinici).
- **Target** (label): la colonna con l'obiettivo da prevedere (es. spam/non-spam, specie del fiore, prezzo dell'immobile).

Dataset Iris – esempio classico

150 fiori (istanze), 4 feature (*sepal length*, *sepal width*, *petal length*, *petal width*), 1 target (specie: *setosa*, *versicolor*, *virginica*). Dimensioni: 150×5 .

Il problema della dimensionalità

Con dataset da 10 o 100 feature, è impossibile per l'occhio umano visualizzare i dati. Si usano algoritmi di **riduzione della dimensionalità** come **t-SNE** (t-distributed Stochastic Neighbor Embedding), che comprimono le feature in 2–3 dimensioni mantenendo le relazioni di vicinanza, permettendo di individuare visivamente cluster o anomalie. Altre alternative: **PCA (Principal Component Analysis)** (lineare, più veloce), **UMAP (Uniform Manifold Approximation and Projection)** (non lineare, più scalabile).

3.7 Famiglie di Machine Learning

3.7.1 Classificazione per supervisione

1. **Apprendimento Supervisionato**: i dati di training includono le etichette (label). L'algoritmo impara a mappare input $\rightarrow$ output.

- **Classificazione**: predice categorie discrete.
 - *Binaria*: spam/non-spam, gatto/non-gatto, maligno/benigno.
 - *Multiclasse*: riconoscimento cifre scritte a mano (MNIST: 10 classi), classificazione specie di iris (3 classi).
- **Regressione**: predice valori continui, come il costo di un'abitazione in base alla posizione geografica.
 - *Interpolazione*: stima di un valore interno al range dei dati (es. dati 1980–2025, stimo il 1983).
 - *Estrapolazione*: stima di un valore esterno al range dei dati (es. dati fino al 2025, prevedo il 2026 – più rischiosa).

2. **Apprendimento Non Supervisionato**: i dati *non* hanno etichette. Il sistema deve scoprire strutture da solo.

- **Clustering**: raggruppa istanze simili – k-means, DBSCAN (es. segmentazione clienti in un e-commerce in base agli acquisti).
- **Riduzione dimensionale**: PCA, t-SNE, UMAP.

- **Anomaly detection:** Isolation Forest, Autoencoder (es. rilevazione frodi su carte di credito).
 - **Association rule learning:** Apriori – es. chi compra salsa barbecue e patatine tende anche ad acquistare bistecca → posizionamento dei prodotti nel supermercato.
3. **Semi-supervisionato:** pochi dati etichettati + molti non etichettati. Es. Google Photos riconosce automaticamente che la persona A appare nelle foto 1, 5, 11 (clustering), poi basta aggiungere *un'unica* etichetta per nome e il sistema nomina tutti i volti.
 4. **Self-supervised:** le etichette vengono generate automaticamente dai dati stessi. Es. mascherare una porzione di un'immagine e addestrare il modello a ricostruirla – le immagini mascherate sono gli input, le originali sono i target. Questa tecnica è alla base dei moderni LLM (previsione del token successivo).
 5. **Reinforcement Learning:** un **agente** interagisce con un ambiente, riceve **premi** (per azioni corrette) o **penalità** (per errori), e impara una **policy** – la strategia che massimizza il premio cumulativo nel tempo.

Esempi di Reinforcement Learning

AlphaGo (DeepMind) ha battuto il campione mondiale di Go analizzando milioni di partite e poi giocando contro se stesso. Durante la partita contro il campione, l'apprendimento era disattivato: AlphaGo applicava la policy appresa.

6. **Ensemble Learning:** più modelli lavorano in combinazione – il risultato finale è più robusto di qualsiasi modello singolo.
 - **Bagging:** addestra modelli paralleli su sottoinsiemi casuali del training set e aggrega le previsioni (es. Random Forest).
 - **Boosting:** addestra modelli in sequenza, dove ognuno cerca di correggere gli errori del precedente (es. XGBoost, AdaBoost).
 - **Stacking:** un meta-modello combina le previsioni dei modelli base.

3.7.2 Batch vs. Online Learning

- **Batch learning:** il modello viene addestrato su *tutti* i dati disponibili in una volta sola, poi viene rilasciato in produzione *senza ulteriore apprendimento*. La performance decade nel tempo (*model rot* o *data drift*) perché il mondo evolve mentre il modello rimane fisso. Soluzione: re-training periodico. Svantaggio: costoso in tempo e risorse per grandi dataset.

Model rot

Un classificatore di immagini gatto/cane addestrato nel 2018 non “muterà” le sue prestazioni per via della biologia dei gatti. Ma se le fotocamere cambiano formato, risoluzione, o le persone iniziano a vestire i propri animali, le prestazioni peggiorano – un esempio di data drift lento.

- **Online learning:** il modello viene aggiornato in modo *incrementale* man mano che arrivano nuovi dati, individualmente o a piccoli gruppi (*mini-batch*). Utile per:
 - Sistemi che devono adattarsi rapidamente (mercati finanziari).
 - Dataset enormi che non entrano in memoria (*out-of-core learning*).
 - Risorse limitate (smartphone, rover su Marte).

Il parametro chiave è il **learning rate**: alto $\rightarrow$ adattamento rapido ma dimentica i vecchi pattern; basso $\rightarrow$ più stabile ma più lento ad adeguarsi.

Rischio dell'online learning

Se arrivano dati di cattiva qualità (sensore difettoso, attacco avversariale al sistema), le prestazioni possono degradarsi rapidamente. È fondamentale monitorare le performance e avere la possibilità di sospendere l'apprendimento e ripristinare uno stato precedente.

3.7.3 Instance-Based vs. Model-Based Learning

- **Instance-based**: il sistema “memorizza” gli esempi di training e confronta i nuovi casi con quelli memorizzati tramite una misura di similarità. Esempio classico: **k-Nearest Neighbors** (kNN) – classifica un nuovo punto in base alla classe di maggioranza tra i k esempi più vicini nello spazio delle feature.
- **Model-based**: costruisce un modello esplicito che cattura i pattern nei dati, poi lo usa per fare previsioni. Esempio: regressione lineare – si impara una relazione parametrica tra input e output che può generalizzare a nuovi dati mai visti.

3.8 La Regressione Lineare

La **regressione lineare** non è un algoritmo di classificazione, ma di *apprendimento supervisionato per valori continui* (apprendimento lineare). Supponiamo di avere dati con un trend di crescita, ad esempio la soddisfazione di vita in funzione del PIL pro capite per diversi paesi. I dati non si disporranno mai perfettamente su una linea retta, ma è necessario misurare la tendenza tenendo conto degli errori.

$$\text{soddisfazione} = \theta_0 + \theta_1 \cdot \text{PIL}_{\text{pro capite}} \quad (3.1)$$

Questa formula è l'equazione della retta $y = q + mx$, dove:

- $\text{PIL}_{\text{pro capite}}$ è la nostra variabile indipendente x ;
- θ_1 rappresenta la pendenza (m);
- θ_0 rappresenta l'intercetta, ossia l'ordinata all'origine (q).

Come si sceglie la retta migliore? L'indice di qualità si basa sulla retta che si avvicina di più in assoluto ai dati sperimentali (*best fit*). Si vuole che la somma delle distanze (residui) tra la retta e i dati reali sia la più piccola possibile. Per ottenere ciò si minimizza la **funzione di costo MSE** (Mean Squared Error – errore quadratico medio), che misura la distanza media al quadrato tra le previsioni del modello e i valori reali:

$$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

La minimizzazione si ottiene calcolando la derivata parziale rispetto a ciascun parametro e ponendola uguale a zero.

Capitolo 4

Dati, Modelli e i Loro Limiti

Lezione 4 – 16/03/2026 – Géron, Cap. 1 (cont.)

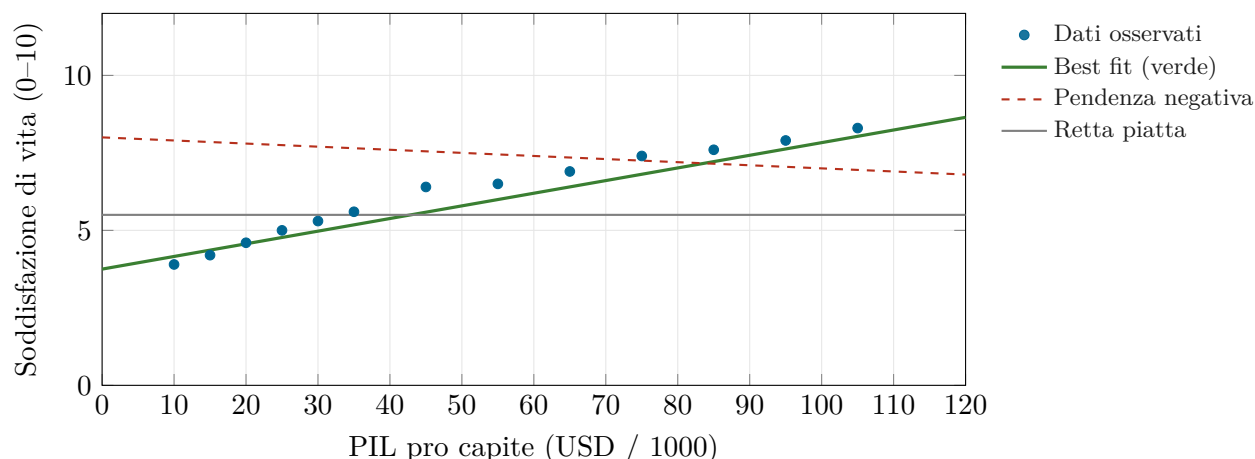
4.1 Ricapitolo: la regressione lineare e la matematica del modello

La **regressione lineare** è più antica sia del Machine Learning che dell'Intelligenza Artificiale. Se ci troviamo di fronte a un problema lineare, i dati possono essere rappresentati tramite una tendenza (trend). Mettendo in relazione il PIL pro capite con la soddisfazione di vita, si osserva che all'aumentare del PIL la soddisfazione cresce – comportamento linearmente crescente.

Ricordiamo l'equazione del modello (già vista nella lezione precedente):

$$\hat{y} = \theta_0 + \theta_1 \cdot x$$

che non è altro che l'equazione classica della retta $y = q + mx$, con θ_0 intercetta e θ_1 pendenza. Il grafico seguente mostra tre possibili rette sugli stessi dati:



- **Verde (pendenza negativa):** più un paese è ricco, meno le persone sarebbero soddisfatte – contraddice i dati.
- **Rosso (retta piatta):** nessuna relazione tra PIL e soddisfazione – inutile.
- **Blu (best fit):** segue il trend ascendente dei dati sperimentali. Non passa per ogni singolo punto, ma rappresenta la **tendenza generale migliore**.

4.1.1 Come si trovano i parametri ottimali?

I parametri θ_0 e θ_1 si identificano minimizzando una **funzione di costo**, tipicamente l'**MSE** (Mean Squared Error), che misura la distanza media al quadrato tra le previsioni del modello e i valori reali. Minore è l'errore complessivo, migliore è l'approssimazione.

Analogia matematica

Come nei problemi di ottimizzazione del liceo – trovare il rettangolo inscritto in una circonferenza con area massima, o il triangolo circoscritto con area minima – si calcolano le soluzioni ottime ponendo a zero la derivata prima. Nel caso della regressione lineare, essendo un problema con 2 variabili (θ_0, θ_1), si calcolano **2 derivate parziali** e si pongono uguali a zero.

Tuttavia, questa è una soluzione di **compromesso**: forzare la retta a passare per un punto specifico altera le distanze da tutti gli altri punti.

Questo problema ha due soluzioni algoritmiche:

- **Normal Equation**: soluzione analitica chiusa – calcola direttamente i parametri ottimali con un'operazione matriciale. Efficiente per dataset piccoli.
- **Gradient Descent (discesa del gradiente)**: soluzione iterativa – parte da parametri casuali e li aggiorna gradualmente nella direzione che riduce l'errore.

Con i parametri ottimali $\theta_0 = 3,75$ e $\theta_1 = 6,78 \times 10^{-5}$, la retta risultante è:

$$\text{soddisfazione} = 3,75 + 6,78 \times 10^{-5} \cdot \text{PIL}$$

Esempio numerico: fissare la soddisfazione e trovare il PIL

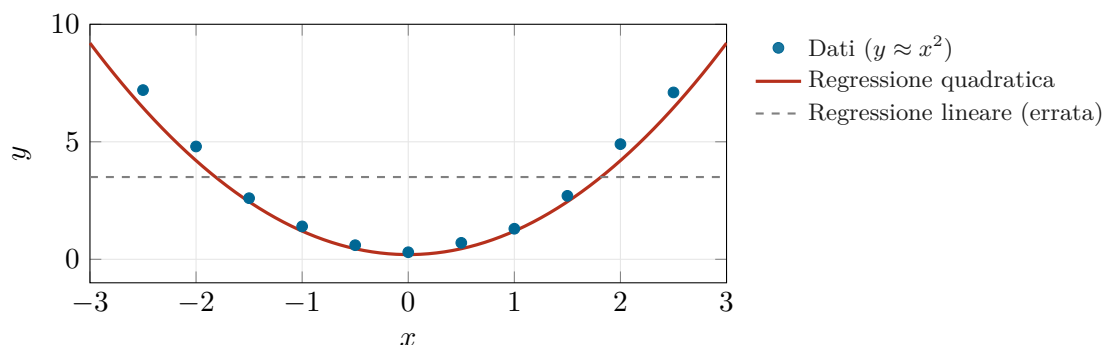
Voglio scoprire quale PIL pro capite corrisponde a una soddisfazione di vita alta ($\hat{y} = 9$):

$$9 = 3,75 + 6,78 \times 10^{-5} \cdot x \quad \Rightarrow \quad x = \frac{9 - 3,75}{6,78 \times 10^{-5}} \approx 77\,434 \text{ USD}$$

Nota: l'asse x del grafico è in USD / 1000, quindi questa scala dilatata è necessaria per evitare che la retta sembri piatta a occhio.

4.1.2 E se il problema NON è lineare?

Se i dati seguono un andamento quadratico ($y = ax^2 + b$), si può usare un **trucco matematico**: sostituire $t = x^2$ e trasformare l'equazione in $y = at + b$, ora lineare. Si risolve con la normale regressione lineare, poi si ritrasforma il risultato.



Più in generale, la **Polynomial Regression** aggiunge feature polinomiali $x^2, x^3, \dots$ e addestra una regressione lineare sullo spazio aumentato. Il rischio è l'**overfitting**: un polinomio di grado troppo alto passa per ogni punto del training set ma generalizza male (vedi sezione 4.5).

4.2 Le 4 fasi di un progetto di Machine Learning



4.3 I dati: quantità, costo e rappresentatività

Se avessimo a disposizione tutti i dati possibili dell'universo (il PIL di tutti i paesi in ogni istante, o tutte le foto del mondo già etichettate), non avremmo bisogno di fare previsioni. Ma la realtà è diversa: è improbabile avere tutti i dati a disposizione, quindi bisogna trovare un **compromesso** tra quantità, qualità e costo computazionale.

4.3.1 Quantità e costo computazionale

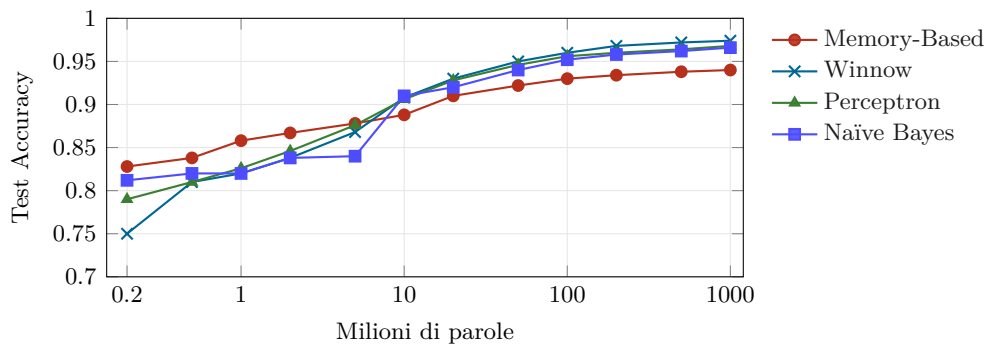
Il dataset **MNIST** è lo standard per la classificazione di cifre scritte a mano: 70 000 immagini, ciascuna una griglia di 28×28 pixel. La matrice risultante ha dimensioni $70\,000 \times 785$ (784 pixel + 1 colonna etichetta). I migliori modelli raggiungono oltre il 99,8% di precisione, ma **mai il 100%**: il fattore umano introduce imperfezioni (un "8" scritto male può sembrare un "9" o un "3") – questo è l'**errore di Bayes irriducibile**.

Avere più dati aumenta l'accuratezza, ma comporta un maggior costo computazionale: il progettista deve trovare un **compromesso** tra quantità dei dati e risorse disponibili.

È meglio un algoritmo complesso o tanti dati? (Banko & Brill, 2001)

Il grafico seguente risponde a una delle domande più antiche dell'IA. Sull'asse x (scala logaritmica) è riportata la quantità di dati usati per l'addestramento; sull'asse y le prestazioni sul test set. Le 4 curve rappresentano 4 diversi algoritmi di classificazione:

- **Scarsità di dati (sinistra del grafico)**: il modello Memory-Based è il migliore; il Winnow è il peggiore.
- **Abbondanza di dati (destra del grafico)**: con l'aumentare dei dati, *tutti* gli algoritmi migliorano e le curve convergono. Il Winnow, da peggiore, diventa il migliore.



4.3.2 Rappresentatività del campione

I dati non devono essere solo tanti – devono **rappresentare il problema reale**:

- **PIL pro capite**: se il training set include solo paesi ricchi o solo paesi poveri, il modello non rappresenterà la realtà globale e avrà prestazioni scarse agli estremi.
- **Classificazione di immagini**: se si addestra il modello solo con foto di gatti, non saprà riconoscere cani o scoiattoli.
- **Studi demografici**: l'età dei campioni deve rispecchiare l'obiettivo. Per studi alimentari serve la popolazione totale; per studi sulla digitalizzazione ha senso restringersi a un range (es. 6–50 anni). I paesi asiatici o africani hanno piramidi demografiche con molti giovani e pochi anziani – questo influisce profondamente sulla composizione del dataset.

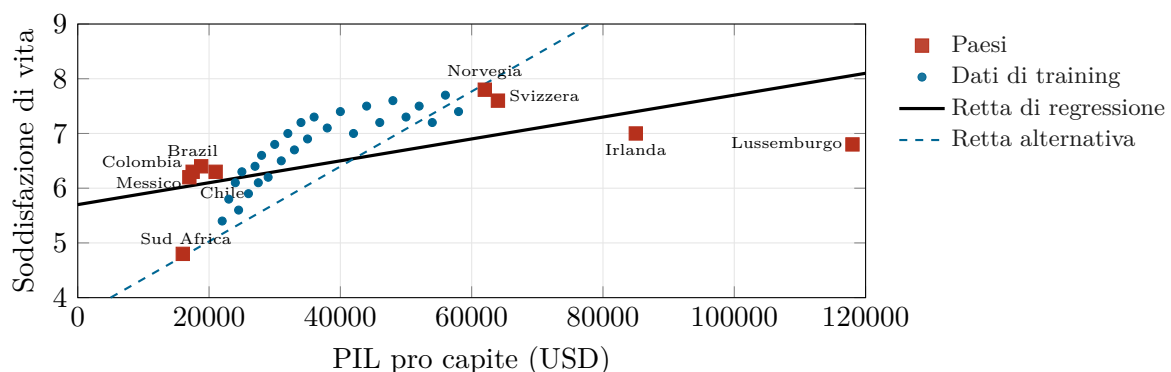
L'aneddoto di Gallup vs. Literary Digest (1936)

Literary Digest: condusse un sondaggio con 10 milioni di persone campionate da elenchi telefonici e abbonamenti a riviste. Predisce Landon vincitore con il 57% dei voti. Roosevelt vinse con il 62%. Errore: **sampling bias** – il campione non era rappresentativo della popolazione.

George Gallup: indovinò il risultato con soli 50000 elettori, ma divisi per età e classe sociale. La **qualità della rappresentatività** contava più della quantità.

4.4 I limiti dei modelli: il problema della non-linearità

La regressione lineare assume dati con andamento rettilineo. Cosa succede se usiamo un modello non lineare (polinomio di grado elevato) per essere più precisi?



- **Quadrati rossi a sinistra:** PIL basso ma soddisfazione sorprendentemente alta (es. Brasile).
- **Quadrati rossi a destra:** PIL alto ma soddisfazione più bassa delle aspettative (es. Lussemburgo).
- **Retta tratteggiata blu:** addestrata solo sui dati parziali (pallini blu). È errata per i paesi anomali – il modello si è *specializzato* su un sottoinsieme e non sa gestire le eccezioni.
- **Retta nera continua:** addestrata su tutti i dati. È una retta di compromesso, passa meno vicino ai pallini, ma fa previsioni più realistiche anche per i casi anomali.

Questo è l'essenza del problema **overfitting vs. underfitting**.

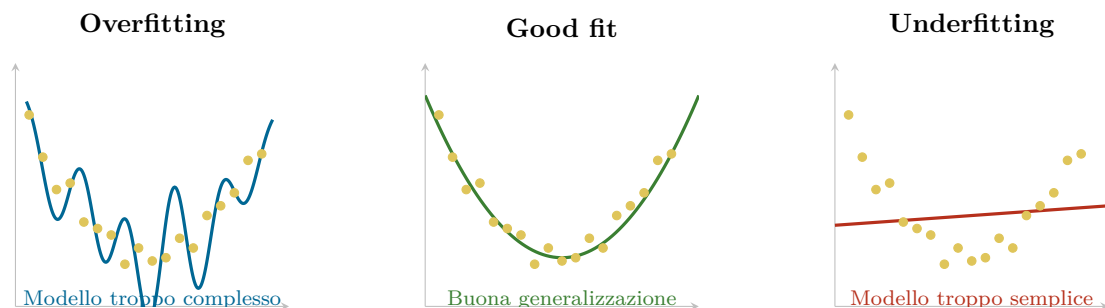
4.5 Overfitting vs. Underfitting

La metafora degli studenti

Tre alunni si preparano per un esame:

1. **Primo:** impara tutti gli esercizi a memoria senza capire.
2. **Secondo:** non studia nulla.
3. **Terzo:** fa gli esercizi richiesti ma capisce la *regola generale*.

Il giorno della verifica, il terzo prenderà il voto più alto. Il primo, di fronte a un problema leggermente diverso da quelli memorizzati, andrà male. Il secondo andrà male sempre.



- **Overfitting (Lo studente che impara a memoria):** il modello memorizza i dettagli e il “rumore” del training set invece di imparare lo schema generale. Ottima corrispondenza in training, errore altissimo sui nuovi dati in test. Causa tipica: modello troppo complesso rispetto alla quantità e alla rumorosità dei dati.

Soluzioni:

- **Regolarizzazione** (L1/Lasso, L2/Ridge): inserisce vincoli matematici per evitare grandi oscillazioni della curva e mantenere i parametri piccoli e semplici.
- Raccogliere più dati di training.
- Ridurre la complessità del modello (meno parametri, meno feature).
- *Early stopping*: interrompere l’addestramento quando l’errore sul validation set smette di migliorare.
- **Underfitting (Lo studente che non studia):** il modello è troppo semplice per catturare la struttura dei dati. Errori elevati sia in training che in test.

Soluzioni:

- Scegliere un modello più espressivo (più parametri, architettura più complessa).
- Feature engineering: aggiungere feature più informative.
- Ridurre la regolarizzazione se è troppo forte.
- Aumentare il numero di epoche di training.

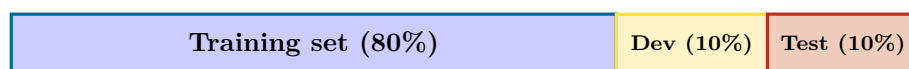
L'Errore di Generalizzazione è proprio la misura dell'errore che il modello commette quando affronta nuovi dati (testing set) estratti dalla stessa distribuzione dei dati usati nel training.

4.6 Divisione del dataset e Cross-Validation

4.6.1 Suddivisione standard (2 vie)

- **Training set** ($\sim 80\%$): per addestrare il modello (trovare θ_0, θ_1 , i pesi, ...).
- **Test set** ($\sim 20\%$): per valutare quanto il modello funziona su dati mai visti.

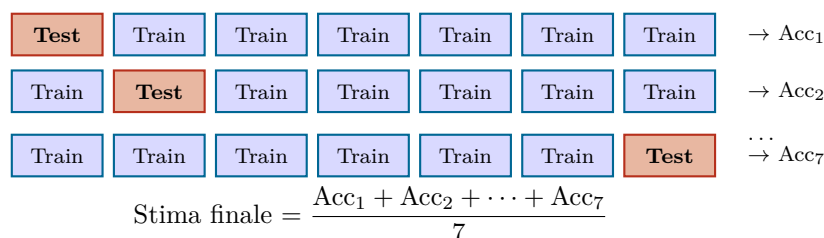
4.6.2 Suddivisione industriale (3 vie)



- **Training set** (80%): addestramento del modello.
- **Dev / Validation set** (10%): usato *in corso d'opera* per confrontare modelli diversi e mettere a punto gli iperparametri – senza toccare il test set.
- **Test set** (10%): usato *solo alla fine*, una volta sola, per la validazione assoluta del modello scelto.

4.6.3 Cross-Validation (Convalida Incrociata)

Se il dataset ha 70 000 immagini, le dividiamo in 7 “pacchetti” da 10 000 (*fold*). A ogni iterazione si usano 60 000 per il training e 10 000 per il test, ruotando il fold di test. Si ottengono 7 valori di precisione; la **media** di questi 7 valori è la stima finale del modello.



- **Vantaggio**: evita bias involontari (es. escludere donne, anziani o un gruppo demografico dal test per puro caso); garantisce che il modello si alleni su diverse combinazioni di dati.
- **Svantaggio**: aumenta il costo computazionale di un fattore k (qui 7). La variante **Stratified k -fold** mantiene le proporzioni delle classi in ogni fold – fondamentale per dataset sbilanciati.

4.7 Iperparametri vs. parametri

- **Parametri:** appresi automaticamente durante il training (es. θ_0, θ_1 nella regressione lineare, i pesi delle reti neurali). Il modello li ottimizza minimizzando la funzione di costo.
- **Iperparametri:** scelti *prima* del training e rimangono fissi durante l'addestramento (es. k in kNN, `max_depth` negli alberi decisionali, il *learning rate*, la forza della regolarizzazione). Non vengono appresi dall'algoritmo, ma impostati dal progettista.

La ricerca degli iperparametri migliori si fa su un **validation set**, usando tecniche come:

- **Grid search:** si prova ogni combinazione di una griglia di valori pre-definiti.
- **Random search:** si campionano combinazioni casuali – spesso più efficiente della grid search per spazi ad alta dimensionalità.
- **Bayesian optimization:** usa un modello probabilistico per guidare la ricerca verso le regioni più promettenti.

4.8 No Free Lunch Theorem

Teorema No Free Lunch (Wolpert, 1996)

Mediando su tutti i possibili problemi, qualsiasi coppia di algoritmi di apprendimento ha la stessa prestazione attesa. **Non esiste un algoritmo universalmente migliore per tutti i problemi.**

In pratica si parte testando alcune famiglie ragionevoli in base al problema (baseline lineare, Random Forest, Gradient Boosting, MLP) e si confrontano in cross-validation.

4.9 Conclusioni sul Machine Learning

Il successo di un sistema ML dipende primariamente da:

1. **Qualità e quantità dei dati:** la collezione e pulizia dei dati rappresenta tipicamente il 70–80% del lavoro reale di un progetto.
2. **Feature engineering:** scegliere le colonne/attributi giusti fa spesso più differenza della scelta dell'algoritmo.
3. **Scelta della famiglia di modelli:** adeguata al problema e alla quantità di dati disponibili.
4. **Rigorosa valutazione:** training/validation/test set ben separati, cross-validation, monitoraggio in produzione.

Capitolo 5

La Classificazione

Lezione 5 – 17/03/2026 – Géron, Cap. 3

5.1 Introduzione alla classificazione

La **classificazione** è la seconda grande famiglia di metodi del Machine Learning supervisionato. A differenza della regressione, che stima valori continui, la classificazione serve a suddividere i dati in **categorie discrete**.

- **Classificazione binaria:** due classi mutuamente esclusive (spam/non-spam, gatto/non-gatto, è un 5 / non è un 5).
- **Classificazione multiclasse:** più classi mutuamente esclusive (cifre 0–9, specie di iris).
- **Classificazione multilabel:** più etichette contemporaneamente per la stessa istanza (es. riconoscimento facciale in una foto di gruppo: Alice sì, Bob no, Charlie sì).
- **Classificazione multioutput:** ogni etichetta può essere multiclasse (es. rimozione del rumore da un'immagine: un'etichetta per pixel, ciascuna da 0 a 255).

5.2 I dataset storici – le “Baseline”

Le **baseline** sono dataset standard e famosi usati dai programmatori per testare e confrontare gli algoritmi. Permettono di avere un riferimento condiviso: se un nuovo algoritmo funziona bene su una baseline, ci si può fidare che funzioni anche altrove.

5.2.1 Iris Dataset (Fisher, 1936)

L'**Iris Dataset** è l'“Hello World” del Machine Learning. Introdotto nel 1936, è composto da:

- **150 istanze:** 150 fiori di iris analizzati in totale.
- **3 classi (target):** le tre specie di iris da classificare:
 - *Iris Setosa*: caratterizzata da dimensioni relativamente piccole.
 - *Iris Versicolor*: dimensioni moderate, a metà strada tra le altre due.
 - *Iris Virginica*: generalmente la più grande tra le tre specie.

- **4 feature:** per ogni fiore sono misurate quattro variabili (in centimetri): lunghezza e larghezza del sepal, lunghezza e larghezza del petalo.
- **Distribuzione bilanciata:** 50 fiori per specie.

È importante nel ML per la sua semplicità (solo 4 attributi), per l'uso come benchmark per confrontare algoritmi diversi, e per studiare l'impatto delle feature su un dataset limitato. In Python è integrato nativamente in `scikit-learn`.

5.2.2 MNIST Dataset (LeCun et al., 1998)

Il **MNIST** (Modified National Institute of Standards and Technology) è un dataset molto più complesso, composto da 70 000 immagini di cifre manoscritte (0–9). Ogni immagine è una griglia di 28×28 pixel, per un totale di 784 pixel. Le immagini sono in scala di grigi: ogni pixel ha un valore numerico da 0 (bianco) a 255 (nero).

Come “vede” la macchina una cifra

Il software non ha gli occhi per vedere la “forma” del numero: analizza una sequenza di 784 numeri (i valori di pixel) e cerca schemi ricorrenti che identifichino la cifra rappresentata. Il dataset è organizzato in una matrice $70\,000 \times 785$ (784 pixel + 1 colonna etichetta).

Curiosità storica: i Babilonesi scrivevano il 3 con tre linee orizzontali e il 2 con due linee. Gli unici numeri che sono rimasti visivamente “fedeli” alla loro logica originaria sono lo 0 e l'1.

Split ufficiale: **60 000 immagini** per il training, **10 000** per il test. Il training set è già mescolato casualmente – questo garantisce che tutti i fold della cross-validation siano simili e che il modello non sia sensibile all'ordine delle istanze.

Scikit-learn e il numero 42

Scikit-learn (sklearn) è la libreria Python per eccellenza per il ML: contiene algoritmi pronti all'uso e generatori di dataset di prova (permette di generare nuvole di punti artificiali con un certo livello di rumore per testare se un algoritmo riesce a separarle in classi).

Il **numero 42** compare quasi universalmente come *random seed* nei programmi ML: è una citazione alla saga *Guida Galattica per Autostoppisti*, in cui un super-calcolatore, dopo milioni di anni di calcoli, stabilisce che 42 è “la risposta alla domanda fondamentale sulla vita, l'universo e tutto quanto”. Fissare il seed garantisce la **riproducibilità** dei risultati.

5.3 Rappresentatività e campionamento stratificato

Uno dei parametri cruciali per la qualità dei dati è la loro **rappresentatività**. Quando si divide un dataset (per la cross-validation o per il train/test split), le **proporzioni originali delle classi** devono essere mantenute in ogni sottoinsieme.

Se nel dataset completo i “5” sono il 10% delle immagini, anche in ogni fold di training e test ci deve essere circa il 10% di “5”. Questo processo si chiama **campionamento stratificato** (*stratified sampling*).

5.4 Addestrare un classificatore binario

Riduciamo MNIST a un problema binario: “È un 5?”. Le etichette diventano **True** (è un 5) o **False** (non è un 5):

L'algoritmo scelto è lo **SGDClassifier** (*Stochastic Gradient Descent Classifier*): un classificatore lineare addestrato con la discesa del gradiente stocastica. È molto efficiente su dataset grandi perché processa le istanze una alla volta (aggiornamento incrementale), ed è adatto all'*online learning*.

5.5 Metriche di qualità

5.5.1 Perché l'accuratezza da sola non basta

La valutazione di un classificatore è significativamente più complessa di quella di un regressore. L'**accuratezza** sembra la metrica naturale:

$$\text{Accuratezza} = \frac{\text{numerodiprevisionicorrette}}{\text{numerototalediprevisioni}} = \frac{TP + TN}{TP + TN + FP + FN}$$

ma può essere ingannevole con dataset **sbilanciati**.

- se il classificatore è perfetto (solo TP e TN) allora l'accuratezza è 1 (100%)
- se il classificatore sbaglia tutto (solo FP e FN) allora l'accuratezza è 0

Il paradosso del classificatore stupido

Nel dataset MNIST, solo circa il 10% delle immagini sono “5”. Un classificatore banale che risponde *sempre* “non è un 5” ottiene il **90% di accuratezza** – senza aver imparato nulla di utile. L'accuratezza da sola non è sufficiente per valutare un classificatore su classi sbilanciate. Servono metriche più informative: **precisione, recall, F1**.

5.5.2 Matrice di confusione

La **matrice di confusione** è lo strumento fondamentale per valutare un classificatore: conta quante volte le istanze di ogni classe reale vengono predette come ogni possibile classe.

		Valori Reali	
		Positivo (è 5)	Negativo (non è 5)
Valori Predetti	Pos. (5)	TP – Vero Positivo Reale: 5 Predetto: 5 ✓ Corretto	FP – Falso Positivo Reale: non 5 Predetto: 5 × Falso allarme
	Neg.	FN – Falso Negativo Reale: 5 Predetto: non 5 × Svista	TN – Vero Negativo Reale: non 5 Predetto: non 5 ✓ Corretto

- Un **TP mancato** diventa un FN (svista).
- Un **TN mancato** diventa un FP (falso allarme).

5.5.3 Precisione

La **precisione** risponde alla domanda: “dei campioni che il modello ha classificato come positivi, quanti erano davvero positivi?”. Penalizza i **falsi allarmi** (FP).

$$P = \frac{TP}{TP + FP} \quad (5.1)$$

Alta precisione $\Rightarrow$ pochi falsi positivi. Utile quando un falso allarme è costoso.

Filtro spam: alta precisione

Un'email importante classificata come spam è un disastro. Preferiamo lasciare passare qualche spam (basso recall) piuttosto che bloccare email legittime. Il costo del FP (email legittima persa) è molto alto.

5.5.4 Recall (Sensibilità)

Il **recall** (o *sensibilità* / *True Positive Rate*) risponde alla domanda: “dei campioni che erano davvero positivi, quanti ne abbiamo recuperati?”. Penalizza le **sviste** (FN).

$$R = \frac{TP}{TP + FN} \quad (5.2)$$

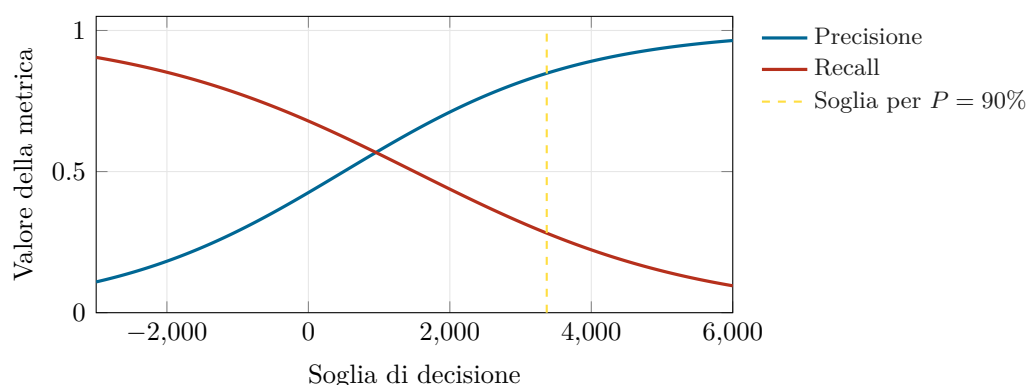
Alto recall $\Rightarrow$ pochi falsi negativi. Utile quando perdere un positivo è costoso.

Screening tumorale: alto recall

Nessun paziente malato deve essere classificato come sano (FN = tumore non rilevato). Accettiamo molti falsi positivi (pazienti sani rimandati per ulteriori esami): il costo del FP è basso, il costo del FN è altissimo.

5.5.5 Il trade-off Precisione – Recall

Precisione e recall sono in **tensione**: aumentare l'una tende a ridurre l'altra. Questo dipende dalla **soglia di decisione** (*threshold*) del classificatore.



- **Alzare la soglia** → il classificatore è più “selettivo” nel dichiarare positivo: **precisione aumenta, recall diminuisce**.
- **Abbassare la soglia** → il classificatore è più “permissivo”: **recall aumenta, precisione diminuisce**.

5.5.6 F1-Score

Quando si vuole una **singola metrica** che bilanci precisione e recall (utile per confrontare due classificatori), si usa l'**F1-Score**: la *media armonica* tra P e R .

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} = \frac{2TP}{2TP + FP + FN} \quad (5.3)$$

La **media armonica** penalizza fortemente i valori bassi: F_1 è alto *solo se entrambi* P e R sono alti. Se uno dei due è basso, l' F_1 crolla.

Scenario	Precisione	Recall	F1
Entrambi alti	90%	85%	87,4%
Uno alto, uno basso	95%	20%	33,1%
Entrambi medi	70%	70%	70,0%

F1 sull'SGD 5-detector reale

$$F_1 = 2 \times (0,837 \times 0,651) / (0,837 + 0,651) \approx 0,733$$

Il classificatore ha una precisione accettabile ma un recall basso – solo il 65% dei “5” viene rilevato. L' F_1 di 73% riflette questo squilibrio.

In sintesi: quale metrica scegliere?

Nessuna singola metrica racconta tutta la storia. Il principio guida è *pensare al costo* di FN vs. FP nel dominio applicativo:

- **Sanità / sicurezza**: privilegiare il **recall** (nessun caso positivo deve sfuggire).
- **Spam / content filtering**: privilegiare la **precisione** (nessun falso allarme).
- **Confronto generale tra classificatori**: usare l'**F1** come metrica di compromesso.
- **Dataset sbilanciati**: evitare l'accuratezza da sola; preferire F_1 , PR-curve, o ROC-AUC.

Capitolo 6

Trade-off P/R, Curva ROC e Class. Multiclasse

Lezione 6 – 23/03/2026 – Géron, Cap. 3 (cont.)

6.1 Ricapitolo: il compromesso Precisione – Recall

Nella lezione precedente abbiamo visto che:

$$P = \frac{TP}{TP + FP} \quad R = \frac{TP}{TP + FN} \quad F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Esiste un **trade-off** fondamentale: aumentare la precisione abbassa il recall e viceversa. La ragione è la **soglia di decisione** τ : i classificatori come `SGDClassifier` producono un **decision score** per ogni istanza, e la soglia decide la classe predetta.

$$\hat{y} = \begin{cases} +1 & \text{se } \text{score}(x) \geq \tau \\ -1 & \text{se } \text{score}(x) < \tau \end{cases}$$

- **Alzare** τ (più esigenti nel dichiarare positivo): la precisione aumenta, il recall diminuisce – meno istanze dichiarate positive, ma quelle dichiarate sono più affidabili.
- **Abbassare** τ (più permissivi): il recall aumenta, la precisione diminuisce – il modello cattura più veri positivi ma anche più falsi allarmi.

Caso A – Video per bambini: alta precisione

Il modello deve bloccare contenuti violenti. Non vogliamo nessun FP (un video violento classificato come sicuro). Per garantirlo, il modello diventa iperprotettivo e scarta anche cartoni innocui – generando molti FN.

Risultato: Alta Precisione, Basso Recall.

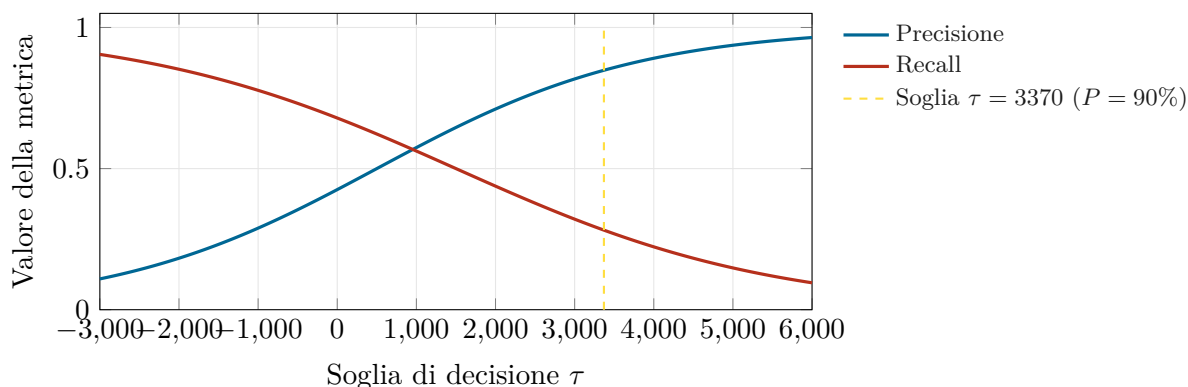
Caso B – Telecamere anti-taccheggio: alto recall

Nessun ladro deve sfuggire: nessun FN ammesso. La telecamera suonerà anche quando un cliente innocente mette le mani in tasca per prendere il telefono – generando molti FP.

Risultato: Alto Recall, Bassa Precisione.

6.2 La curva Precisione-Recall e come trovare la soglia ottimale

Per visualizzare il trade-off su *tutte* le soglie possibili, si usa la **curva Precisione-Recall (PR curve)**, ottenuta con `precision_recall_curve()` in Scikit-learn.



Le due curve non sono in concordanza: se una sale, l'altra scende.

La curva P vs. R

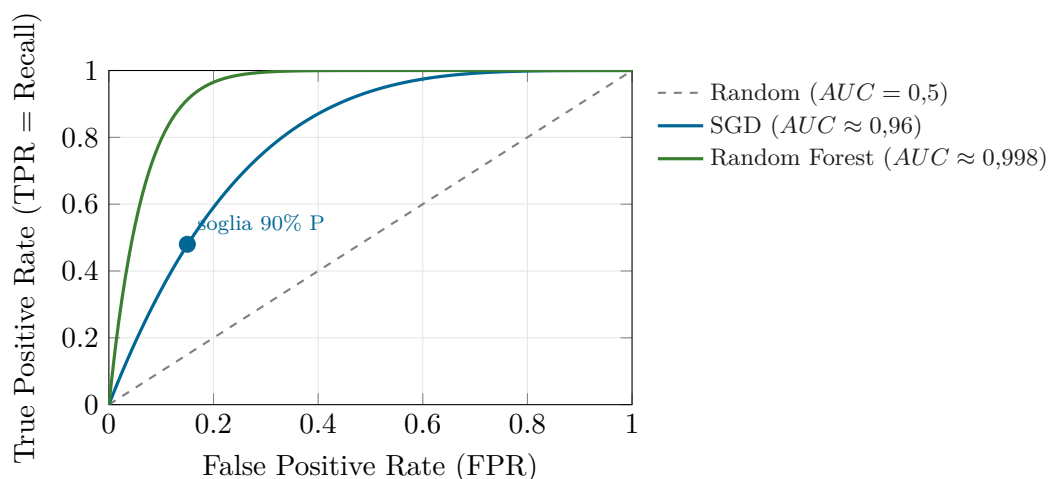
Un modo alternativo di visualizzare il trade-off è tracciare la **Precisione direttamente contro il Recall** (invece che entrambe contro la soglia). La curva ideale è vicina all'angolo in alto a destra. La precisione tende a calare bruscamente quando il recall supera circa l'80% – lì conviene scegliere il punto operativo.

6.3 La curva ROC e l'AUC

La curva **ROC** (*Receiver Operating Characteristic*) è un altro strumento per valutare i classificatori binari. Traccia il **True Positive Rate** (recall) contro il **False Positive Rate**:

$$TPR = \frac{TP}{TP + FN} \quad FPR = \frac{FP}{FP + TN} = 1 - \text{Specificità}$$

Il FPR è la percentuale di *veri negativi* classificati erroneamente come positivi – i falsi allarmi sui negativi.



AUC – Area Under the Curve

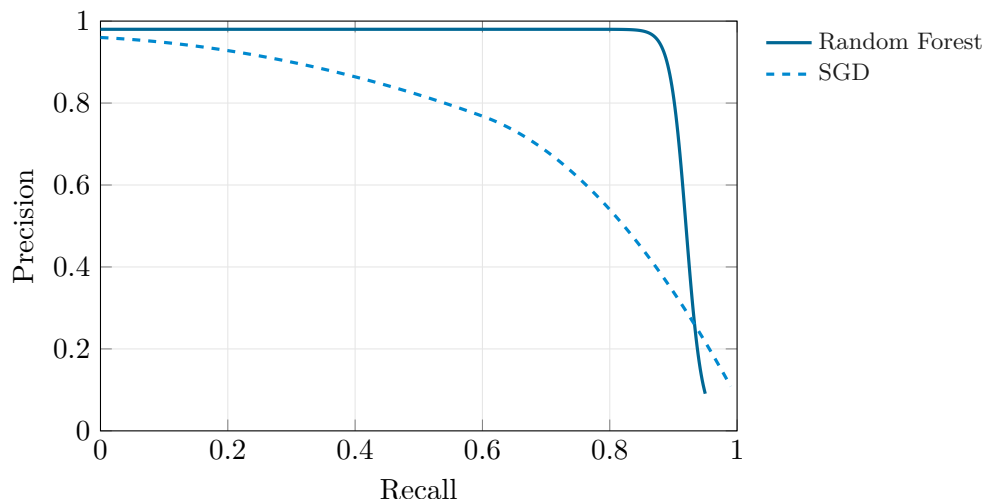
L'**AUC** sintetizza l'intera curva ROC in un numero:

- $AUC = 1,0 \rightarrow$ classificatore perfetto (angolo in alto a sinistra).
- $AUC = 0,5 \rightarrow$ classificatore casuale (diagonale).
- AUC = probabilità che, dato un positivo e un negativo casuali, il modello assegni allo score del positivo un valore più alto. *Indipendente dalla soglia.*

SGD vs. Random Forest sul 5-detector

- **SGDClassifier**: $F_1 \approx 0,733$, $ROC-AUC \approx 0,960$.
- **RandomForestClassifier**: $F_1 \approx 0,924$, $ROC-AUC \approx 0,998$.

Il Random Forest usa `predict_proba()` (probabilità stimate) invece di `decision_function()` perché non ha funzione lineare di decisione. La sua curva PR è molto più vicina all'angolo in alto a destra, confermando la superiorità.

**6.3.1 Quando usare PR curve e quando ROC****PR vs. ROC: regola pratica**

Se la classe positiva è **rara** (dataset sbilanciato) o i falsi positivi sono particolarmente costosi, preferire la **curva PR**: la ROC può sembrare ottima anche con bassa precisione, perché la massa di TN gonfia il TNR. Nel 5-detector (solo 10% di positivi), la curva PR rivela molto più margine di miglioramento rispetto alla ROC.

Metrica	Quando usarla	Quando NON usarla
Accuracy	Classi bilanciate	Classi molto sbilanciate
Precision	FP costoso	Quando mancare positivi è grave
Recall	FN costoso	Quando i FP sono costosi
F1	Compromesso $P-R$	Quando P e R hanno pesi diversi
ROC-AUC	Soglia ignota, classi bilanciate	Dati molto sbilanciati
PR-AUC	Dataset sbilanciati, FP costosi	Quando i negativi sono rilevanti

6.4 Classificazione Multiclasse

Finora abbiamo lavorato in modalità binaria (“è un 5 / non è un 5”). Cosa succede se vogliamo classificare *tutte e 10* le cifre?

Alcuni algoritmi sono **nativamente multiclasse**: Random Forest, Naive Bayes, KNN, MLP, Regressione Logistica. Altri sono **intrinsecamente binari**: SVM, SGD. Per usarli su più classi esistono due strategie.

6.4.1 One-vs-Rest (OvR)

Si addestrano N classificatori binari, uno per classe: “zero vs. tutti”, “uno vs. tutti”, $\dots$, “nove vs. tutti”. Per classificare una nuova istanza, si eseguono tutti gli N classificatori e si sceglie la classe il cui classificatore produce il **punteggio più alto**.

- Costo: $O(N)$ classificatori, ognuno addestrato sull'intero training set.
- Default di Scikit-learn per la maggior parte degli algoritmi binari (es. SGD).

6.4.2 One-vs-One (OvO)

Si addestra un classificatore per **ogni coppia** di classi: “0 vs. 1”, “0 vs. 2”, $\dots$, “8 vs. 9”. Il numero totale di classificatori è:

$$\binom{N}{2} = \frac{N(N-1)}{2}$$

Per MNIST con 10 classi: $\frac{10 \times 9}{2} = 45$ classificatori. Per classificare, si eseguono tutti i 45 e si conta chi vince più “duelli” (*majority vote*).

- Vantaggio: ogni classificatore si allena solo su 2 classi $\rightarrow$ dataset molto più piccoli, adatto ad algoritmi che scalano male (come SVM).
- Svantaggio: costo quadratico $O(N^2)$ nel numero di classificatori.
- Default di Scikit-learn per SVM.

6.5 Analisi degli errori

Una volta selezionato un modello promettente, l'**analisi degli errori** è il passo successivo per capire come migliorarlo.

6.5.1 Matrice di confusione multiclasse, normalizzata

Con 10 classi la matrice diventa 10×10 . Normalizzarla **per riga** (dividendo per il numero di istanze di ogni classe reale) permette di vedere le **percentuali** invece dei valori assoluti – più utile quando le classi hanno numerosità diverse.

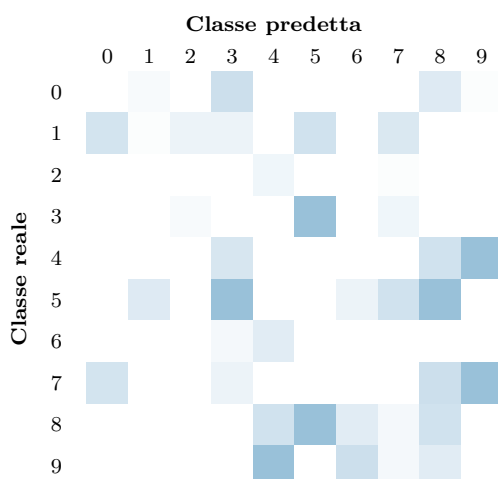
MNIST: cosa sbaglia l'SGD?

Dalla matrice normalizzata emerge che:

- Solo l'**82%** delle immagini di “5” viene classificato correttamente.
- Il **10%** dei “5” viene scambiato per “8”.
- Il **36%** degli errori su “7” sono classificazioni come “9”.

6.5.2 Il trucco della diagonale

Per far risaltare solo gli errori, si **azzerà la diagonale principale** (che contiene i successi) e si visualizza la matrice come *heatmap*. Le celle più luminose corrispondono agli errori più frequenti – così si capisce subito se il modello confonde spesso, ad esempio, il “3” con il “5” o il “4” con il “9”.



6.5.3 Feature Engineering mirata

Dall'analisi emergono strategie concrete:

- **Raccogliere più dati** per le coppie confuse (es. più immagini di cifre simili all'8 che non sono 8).
- **Creare feature discriminative**: aggiungere il conteggio dei cerchi chiusi (8 ne ha 2, 6 ne ha 1, 5 ne ha 0) – questo aiuta a distinguere “5” da “8”.
- **Pre-processing**: centrare le immagini, ridurre la rotazione.
- **Data augmentation**: aggiungere varianti leggermente shiftate/ruotate delle immagini di training per rendere il modello più robusto alla posizione.

6.6 Classificazione Multilabel

In alcuni problemi ogni istanza può avere **più etichette contemporaneamente** – non è una scelta mutualmente esclusiva.

MNIST multilabel

Per ogni cifra definiamo due label:

- **“Grande”**: la cifra è ≥ 7 (True/False).
- **“Dispari”**: la cifra è dispari (True/False).

Per il “5”: output = $[False, True]$ – non è grande, ma è dispari.

Per il “7”: output = $[True, True]$ – è grande e dispari.

Per valutare un classificatore multilabel si calcola l’**F1 per ogni label** e poi si fa la media:

- **average="macro"**: media non ponderata – tutte le label contano uguale.
- **average="weighted"**: media ponderata per il numero di istanze di ogni label – enfatizza le label più frequenti.

6.6.1 ClassifierChain: modellare le dipendenze tra label

I classificatori indipendenti per ogni label ignorano le **correlazioni** tra etichette. Ad esempio, se una cifra è “grande” (≥ 7) ha il doppio delle probabilità di essere dispari. **ClassifierChain** risolve questo passando le previsioni delle label precedenti come feature aggiuntive ai modelli successivi nella catena.

6.7 Classificazione Multioutput

La **classificazione multioutput** è una generalizzazione del multilabel in cui ogni etichetta può avere **più di due valori possibili** (multiclasse per ogni output).

Denoising di immagini MNIST

Input: immagine 28×28 con rumore casuale aggiunto ai pixel.

Output: l’immagine originale pulita – cioè 784 valori interi da 0 a 255, uno per pixel. Questo è un task multioutput: 784 label, ciascuna con 256 valori possibili. Un **KNeighborsClassifier** addestrato su immagini rumorose (input) con le originali come target impara a “ripulire” nuove immagini rumorose restituendo i valori di pixel più probabili.

Il confine tra classificazione e regressione

Nel denoising, predire l’intensità di un pixel (da 0 a 255) potrebbe essere visto come regressione. La distinzione tra classificazione e regressione non è sempre netta: i sistemi multioutput non sono limitati alla classificazione e possono combinare label di classe e valori continui.

Capitolo 7

Alberi di Decisione

Lezione 7 – 30/03/2026 – Géron, Cap. 6

7.1 Introduzione

Gli **Alberi di Decisione** (Decision Trees) sono algoritmi di Machine Learning supervisionato estremamente intuitivi: a differenza di altri modelli, non cercano equazioni matematiche complesse, ma si basano su una serie di **domande a cascata** (if/else). Sono algoritmi versatili, utilizzabili per:

- **Classificazione** (DecisionTreeClassifier): classificare iris, riconoscere cifre, filtrare spam.
- **Regressione** (DecisionTreeRegressor): stimare prezzi, prevedere valori continui.
- Predizioni **multioutput**: più output contemporaneamente.

Richiedono pochissimo preprocessing: nessun *feature scaling* o *centering*. Sono la componente fondamentale delle **Random Forest** e di altri ensemble, tra i modelli più potenti disponibili oggi su dati tabulari.

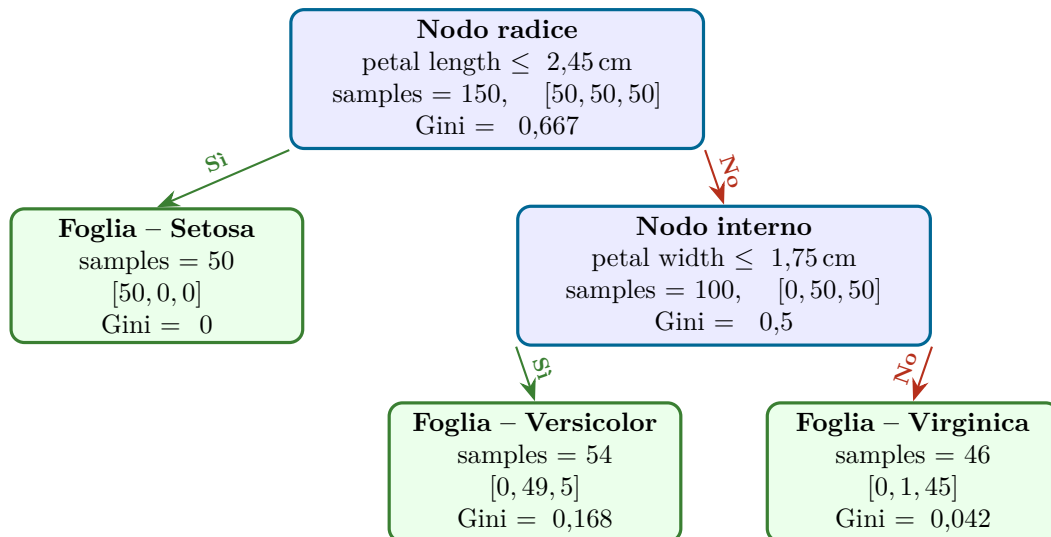
Fanno parte della **xAI (Explainable AI)**:

- **White Box** (scatola trasparente): l'analista capisce perfettamente perché il modello ha preso una decisione – basta leggere la sequenza di if/else.
- **Black Box** (scatola nera): algoritmi come le reti neurali profonde. Funzionano benissimo, ma la logica matematica interna è incomprensibile all'uomo.

L'interpretabilità è cruciale in ambiti regolamentati: *credit scoring*, sanità, sistemi di giustizia, GDPR Art. 22 (diritto a non essere soggetti a decisioni automatizzate non spiegate).

7.2 Come funziona – l'esempio Iris

Usiamo il dataset Iris con solo 2 feature: **lunghezza** e **larghezza del petalo**. L'albero impara a classificare le tre specie (Setosa, Versicolor, Virginica) tramite soglie sulle feature.



Come ragiona l'albero: si parte sempre dal nodo radice e si scende lungo i rami seguendo il valore della feature.

1. **Nodo radice:** “La lunghezza del petalo è $\leq 2,45$ cm?”

- **Sì** → 50 fiori, tutti Setosa [50, 0, 0] → nodo puro, **foglia**.
- **No** → 100 fiori misti [0, 50, 50] → nodo impuro, serve un'altra domanda.

2. **Nodo interno:** “La larghezza del petalo è $\leq 1,75$ cm?”

- **Sì** → 54 fiori, di cui 49 Versicolor [0, 49, 5] → foglia, classe predetta: Versicolor.
- **No** → 46 fiori, di cui 45 Virginica [0, 1, 45] → foglia, classe predetta: Virginica.

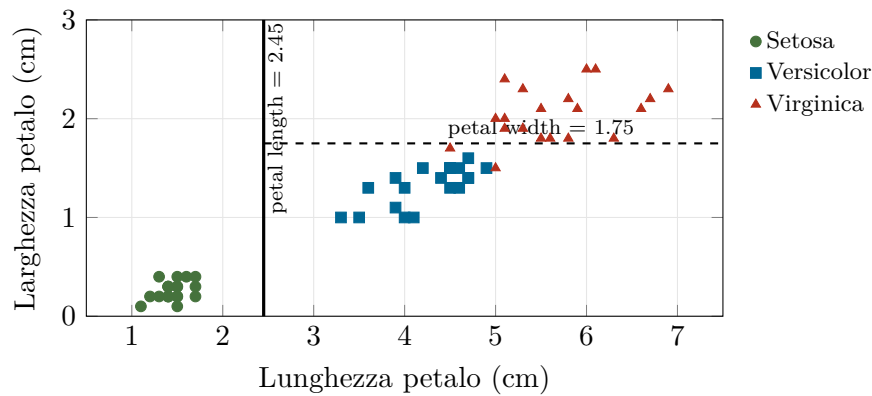
Alla foglia, la **probabilità stimata** è il rapporto istanze-classe/totale: $49/54 \approx 90,7\%$ per Versicolor. La classe predetta è quella con la probabilità più alta.

Spesso i dataset hanno migliaia di attributi

L'albero di decisione ignora automaticamente le feature inutili e seleziona solo quelle davvero discriminanti. Nel caso Iris con 4 feature, l'algoritmo sceglie autonomamente di usare solo lunghezza e larghezza del petalo, ignorando le misure del sepal.

7.2.1 I confini di decisione

Riportando l'albero su un piano cartesiano, i **decision boundary** sono sempre *linee rette orizzontali e verticali* – mai curve o oblique. La linea verticale spessa (depth 0) separa Setosa da tutto il resto. La linea orizzontale tratteggiata (depth 1) separa Versicolor da Virginica. Aumentando **max_depth** si aggiungono ulteriori linee ortogonali – ma si rischia overfitting.



7.3 Metriche di impurità

Come decide l'algoritmo qual è la **domanda migliore** da fare a ogni nodo? Cerca la domanda (feature + soglia) che divide i dati nei gruppi più "puri" possibile. Due metriche:

7.3.1 Indice di Gini

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2 \quad (7.1)$$

dove $p_{i,k}$ è la proporzione della classe k nel nodo i .

- Nodo **puro** (tutti della stessa classe): $G = 0$.
- Nodo completamente **misto** (K classi equiprobabili): $G = 1 - 1/K$. Per 2 classi: massimo $G = 0,5$.

Calcolo di Gini per tutti i nodi dell'albero Iris

Nodo radice [50, 50, 50], totale = 150:

$$G = 1 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 = 1 - 3 \times \frac{1}{9} = 1 - \frac{1}{3} \approx 0,667$$

Nodo Setosa [50, 0, 0]: $G = 1 - 1^2 - 0 - 0 = 0$ (puro)

Nodo depth-1 dx [0, 50, 50]:

$$G = 1 - 0 - \left(\frac{50}{100}\right)^2 - \left(\frac{50}{100}\right)^2 = 1 - 0,25 - 0,25 = 0,5$$

Nodo Versicolor [0, 49, 5], totale = 54:

$$G = 1 - 0 - \left(\frac{49}{54}\right)^2 - \left(\frac{5}{54}\right)^2 = 1 - 0,823 - 0,009 \approx 0,168$$

Nodo Virginica [0, 1, 45], totale = 46:

$$G = 1 - 0 - \left(\frac{1}{46}\right)^2 - \left(\frac{45}{46}\right)^2 \approx 1 - 0,000 - 0,958 = 0,042$$

7.3.2 Entropia di Shannon

$$H_i = - \sum_{k=1}^n p_{i,k} \cdot \log_2 p_{i,k} \quad (7.2)$$

Il concetto di entropia viene dalla termodinamica (misura il disordine) e dalla teoria dell'informazione di Shannon (misura l'imprevedibilità). Nodo puro ($p = 1$), allora $\log_2(1) = 0$ *equindi anche l'entropia* $\Rightarrow H = 0$.

Entropia del nodo Versicolor $[0, 49, 5]$

$$H = -\frac{49}{54} \log_2 \frac{49}{54} - \frac{5}{54} \log_2 \frac{5}{54} \approx -(0,907 \times (-0,141)) - (0,093 \times (-3,426)) \approx 0,445$$

Gini vs Entropia: quale scegliere?

In pratica producono alberi molto simili e con prestazioni analoghe. Differenze:

- **Gini** (default Scikit-learn): leggermente più veloce (nessun calcolo di logaritmo); tende a isolare la classe più frequente in un ramo.
- **Entropia**: produce alberi leggermente più bilanciati.

Gini è un buon punto di partenza. Il criterio si imposta con `criterion="entropy"` in Scikit-learn.

7.4 L'algoritmo CART

Scikit-learn usa l'algoritmo **CART** (Classification and Regression Trees, Breiman 1984) per costruire alberi binari (ogni nodo ha esattamente 2 figli).

Ad ogni passo, CART cerca la coppia (feature k , soglia t_k) che minimizza la funzione di costo:

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \cdot G_{\text{left}} + \frac{m_{\text{right}}}{m} \cdot G_{\text{right}} \quad (7.3)$$

È una **media pesata** dell'impurità dei due nodi figli, dove il peso è la proporzione di istanze. Il processo è ricorsivo: ogni sottoinsieme viene ulteriormente diviso, finché non si raggiunge un criterio di arresto (profondità massima, minimo campioni, ecc.).

7.4.1 CART è un algoritmo greedy

Approccio Greedy

CART prende la decisione **localmente migliore** in ogni nodo, senza guardare avanti di più livelli. Non garantisce di trovare l'albero ottimale globale.

Trovare l'albero perfetto è un problema **NP-Completo** – costo esponenziale $O(e^m)$, intrattabile anche per dataset piccoli. Ci si accontenta dell'ottimo locale trovato dall'approccio greedy.

Confronto con Dijkstra

Dijkstra (algoritmo per i percorsi minimi sui grafi) è anch'esso greedy: “qual è la città inesplorata più vicina? Ci vado!”. La differenza fondamentale: Dijkstra **garantisce** l'ottimo globale (a patto che non ci siano archi con peso negativo). CART **non** garantisce l'ottimo globale – due alberi addestrati sullo stesso dataset con semi diversi possono risultare significativamente diversi.

7.4.2 Costo computazionale

- **Inferenza** (previsione): scendere dall'albero radice a una foglia. Per un albero bilanciato: $O(\log_2 m)$. Velocissimo, indipendente dal numero di feature.
- **Training**: a ogni nodo si confrontano tutte le n feature su tutte le m istanze. Complessità: $O(n \cdot m \log_2 m)$. Scalabile a dataset ragionevolmente grandi.

Esempio pratico

Se il training con $m = 10^6$ istanze richiede 1 ora, con $m = 10^7$ richiederà circa:

$$\frac{10^7 \cdot \log_2(10^7)}{10^6 \cdot \log_2(10^6)} \approx \frac{10 \times 23,25}{19,93} \approx 11,7 \text{ ore}$$

(non 10 ore: il log cresce leggermente con m).

7.5 Regressione con gli alberi di decisione

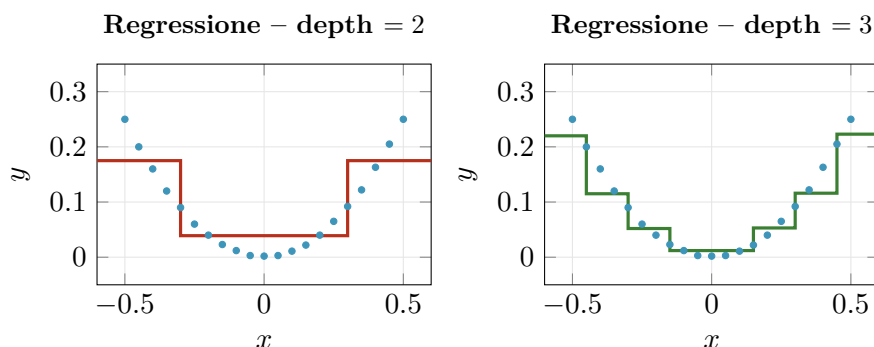
`DecisionTreeRegressor` funziona con la stessa struttura: nodi interni con test su feature, foglie con valori continui. La differenza è nella **funzione di costo**: invece di minimizzare Gini, si minimizza l'MSE ponderato.

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \cdot MSE_{\text{left}} + \frac{m_{\text{right}}}{m} \cdot MSE_{\text{right}} \quad (7.4)$$

dove:

$$\bar{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y_i \quad MSE_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} (y_i - \bar{y}_{\text{node}})^2$$

La previsione in ogni foglia è la **media** dei valori target delle istanze che vi arrivano. Il risultato è una funzione a scalini – più è profondo l'albero, più i gradini sono piccoli.



Anche la regressione è soggetta a overfitting: senza limiti, l'albero crea una foglia per ogni punto del training set, con previsioni perfette ma generalizzazione pessima. La regolarizzazione (es. `min_samples_leaf=10`) risolve il problema.

7.6 Overfitting e regolarizzazione

I decision tree sono modelli **non parametrici**: il numero di parametri non è fissato prima del training, ma cresce con i dati. Senza vincoli, l'albero cresce finché ogni foglia è pura al 100% – overfitting garantito.

Effetto della regolarizzazione – dataset Moons

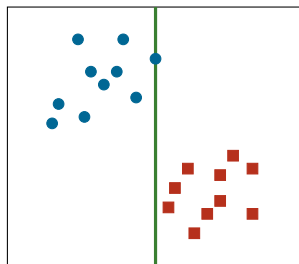
L'albero non regolarizzato produce confini di decisione estremamente irregolari e memorizza il rumore. L'albero con `min_samples_leaf=5` generalizza meglio nonostante un training accuracy leggermente inferiore.

Un approccio alternativo è il **pruning** (potatura): si fa crescere l'albero senza limiti, poi si rimuovono i nodi non significativi usando test statistici (es. test χ^2). Se il miglioramento di purezza non è statisticamente significativo ($p\text{-value} > 5\%$), il nodo è eliminato.

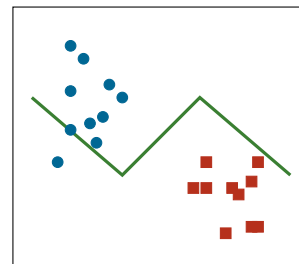
7.7 Sensibilità all'orientamento degli assi

I confini di decisione sono **sempre perpendicolari agli assi delle feature**. Questo rende gli alberi sensibili all'orientamento dei dati: una semplice rotazione del dataset può trasformare un problema banale in uno che richiede molti più split.

Dati originali – facile



Dati ruotati di 45° – difficile



A sinistra, un solo taglio verticale separa perfettamente le due classi. A destra, dopo una rotazione di 45°, il confine naturale è diagonale ma l'albero può usare solo linee ortogonali – servono molti più split, con rischio di overfitting.

Soluzione: pre-processare i dati con **StandardScaler** + **PCA** per ruotare e decorrelare le feature prima di addestrare l'albero.

7.8 Alta varianza e Random Forest

Gli alberi di decisione sono modelli ad **alta varianza**: piccole modifiche nei dati di training o un seed diverso possono produrre alberi strutturalmente molto diversi. L'algoritmo di training di Scikit-learn è stocastico (seleziona casualmente le feature da valutare a ogni nodo) – anche riaddestrando sugli stessi dati si ottiene un albero differente.

Questa alta varianza sembra un difetto, ma negli **ensemble** diventa un punto di forza:

Random Forest

Una **Random Forest** è un ensemble di alberi di decisione, ciascuno addestrato su:

- Un sottoinsieme casuale delle **istanze** (bootstrap sampling – *bagging*).
- Un sottoinsieme casuale delle **feature** a ogni split.

Le previsioni vengono combinate per **voto di maggioranza** (classificazione) o **media** (regressione). Mediando su tanti alberi diversi, la varianza si riduce drasticamente senza aumentare il bias – le foreste generalizzano molto meglio dei singoli alberi.

Dal singolo albero alla foresta – costruzione manuale

1. Generare 1 000 sottoinsiemi del training set, ciascuno con 100 istanze casuali.
2. Addestrare un `DecisionTreeClassifier` su ciascun sottoinsieme.
3. Per ogni istanza del test set, raccogliere le 1000 previsioni e tenere quella di maggioranza (`scipy.stats.mode`).
4. Risultato: accuratezza leggermente superiore al singolo albero addestrato sull'intero training set.

Questo è esattamente ciò che fa `RandomForestClassifier` in modo automatico ed efficiente.

Random Forest e **Gradient Boosting** (XGBoost, LightGBM, CatBoost) – che addestrano alberi in sequenza correggendo gli errori dei precedenti – sono fra i modelli più usati e vincenti su dati tabulari nelle competizioni di ML.

Capitolo 8

Apprendimento Non Supervisionato e K-Means

Lezione 8 – 13/04/2026 – Géron, Cap. 9

8.1 Introduzione

Nella maggior parte delle situazioni reali, i dati non arrivano con una “risposta corretta” o un’etichetta: la colonna target non esiste. In questi casi, possiamo affidarci all’**Apprendimento Non Supervisionato**, che studia come estrarre strutture e pattern intrinseci dai dati *senza* una guida esterna.

La metafora della torta di Yann LeCun

Yann LeCun (Meta AI, Turing Award 2018): “*Se l’intelligenza fosse una torta, l’apprendimento non supervisionato sarebbe la torta, l’apprendimento supervisionato sarebbe la glassa e il Reinforcement Learning sarebbe la ciliegina.*”

La stragrande maggioranza dei dati disponibili nel mondo è **non etichettata**: si pensi a tutto il testo sul web, alle immagini di sorveglianza, ai log di sistema. Etichettare manualmente costa tempo e denaro.

8.1.1 Applicazioni principali

- **Clustering**: raggruppare istanze simili – segmentazione clienti, analisi immagini, sistemi di raccomandazione.
- **Rilevamento di anomalie** (*anomaly detection*): identificare eventi rari o sospetti – frodi bancarie, difetti di produzione, acquisti anomali.
- **Stima della densità**: stimare la distribuzione di probabilità dei dati – usata a sua volta per l’anomaly detection.
- **Riduzione dimensionale**: PCA, t-SNE, UMAP, autoencoder – comprimere le feature preservando l’informazione rilevante.
- **Apprendimento semi-supervisionato**: usare dati non etichettati per migliorare le prestazioni di modelli supervisionati quando le etichette sono poche.

8.2 Classificazione vs. Clustering

	Classificazione (Supervisionata)	Clustering (Non supervisionato)
Etichette nel training	Sì (note a priori)	No
Classi	Definite prima del training	Emergono dai dati
Valutazione	Accuracy, F1, ROC-AUC	Inertia, Silhouette
Esempio	Iris: specie nota, si impara a distinguere	Iris: nessuna specie, si trovano gruppi

Il clustering **non** distingue i nomi delle classi a priori. È l'analista umano che, a posteriori, interpreta e dà un senso ai gruppi emersi. Considera il dataset Iris: a sinistra è un problema di classificazione (specie note), a destra è clustering (nessuna etichetta).

Segmentazione clienti – esempio tipico di clustering

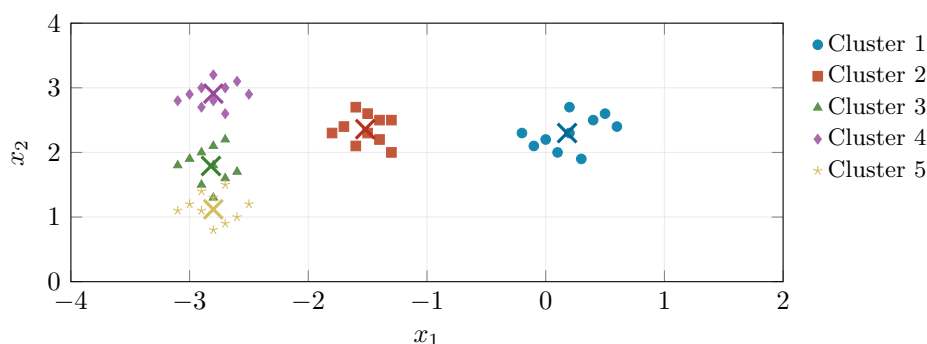
Un e-commerce raggruppa gli acquirenti in base ai prodotti acquistati e all'attività sul sito. L'algoritmo non sa che si tratta di “giovani appassionati di tecnologia” o “adulti interessati al giardinaggio” – è l'analista che dà il nome al cluster. Una volta identificati, ogni segmento può ricevere campagne di marketing personalizzate.

8.3 L'algoritmo K-Means

Il **K-Means** è l'algoritmo di clustering più diffuso. Si basa sul concetto di **centroide**: il baricentro (centro di massa) di un cluster.

8.3.1 Funzionamento passo per passo

1. **Inizializzazione**: si posizionano casualmente k centroidi nello spazio dei dati.
2. **Assegnazione**: ogni istanza viene assegnata al centroide più vicino sfruttando la distanza euclidea $d = \sum_i \sqrt{(x_C - x_i)^2 + (y_C - y_i)^2}$, dove C è il centroide e i sono i punti nella nuvola.
3. **Aggiornamento**: ogni centroide viene spostato alla media delle istanze assegnate.
4. **Ripetizione**: i passi 2–3 si ripetono finché i centroidi smettono di muoversi (*convergenza*).



La convergenza è **garantita matematicamente**: la somma delle distanze quadratiche tra ogni istanza e il suo centroide più vicino (*inerzia*) può solo diminuire ad ogni iterazione e non può essere negativa – quindi converge in un numero finito di passi. In pratica converge molto velocemente, spesso in poche iterazioni.

8.3.2 Tassellazione di Voronoi

Il risultato finale divide lo spazio in **regioni di Voronoi**: ogni punto dello spazio è più vicino al proprio centroide che a qualsiasi altro. I confini tra regioni adiacenti sono i **decision boundary** del modello.

Robustezza di un cluster

Un cluster è **robusto** se i suoi punti sono lontani dai confini della tassellazione. Se un nuovo punto cade lontano dal confine, la sua assegnazione è affidabile. Se cade vicino al confine, è difficile affermare con certezza a quale cluster appartenga – ed è qui che il *soft clustering* è più informativo dell'*hard clustering*.

8.3.3 Hard clustering vs. Soft clustering

- **Hard clustering** (default): ogni istanza appartiene a *esattamente* un cluster.
- **Soft clustering**: si assegna a ogni istanza un **punteggio per ciascun cluster** – la distanza dal centroide, oppure una probabilità. Più informativo, ma meno netto.

In Scikit-learn, `kmeans.transform(X_new)` restituisce la distanza di ogni nuova istanza da ciascun centroide – questo vettore di k distanze può essere usato direttamente come vettore di feature per un modello downstream (**riduzione dimensionale non lineare**).

8.4 Inizializzazione dei centroidi e K-Means++

L'algoritmo non garantisce di trovare l'**ottimo globale**: se i centroidi iniziali sono sfortunati, si può convergere a una soluzione sub-ottimale. Ci sono due strategie principali per mitigare questo rischio. Eseguire K-Means più volte con centroidi iniziali diversi e tenere la soluzione con l'**inerzia** (somma delle distanze quadratiche) più bassa. Introdotto da Arthur & Vassilvitskii (2006), seleziona i centroidi iniziali in modo che siano **distanti tra loro**:

1. Scegliere il primo centroide $c^{(1)}$ casualmente tra le istanze.
2. Per ogni istanza $x^{(i)}$, calcolare $D(x^{(i)}) = \text{distanza dal centroide già scelto più vicino}$.
3. Scegliere il nuovo centroide $c^{(i)}$ con probabilità proporzionale a $D(x^{(i)})^2$: le istanze più lontane dai centroidi già scelti sono molto più probabili di essere scelte.
4. Ripetere i passi 2–3 finché tutti i k centroidi sono stati scelti.

Questo fa sì che i centroidi iniziali siano ben distribuiti, riducendo drasticamente la probabilità di convergere a soluzioni sub-ottimali. **È il default di Scikit-learn.**

8.4.1 Mini-Batch K-Means

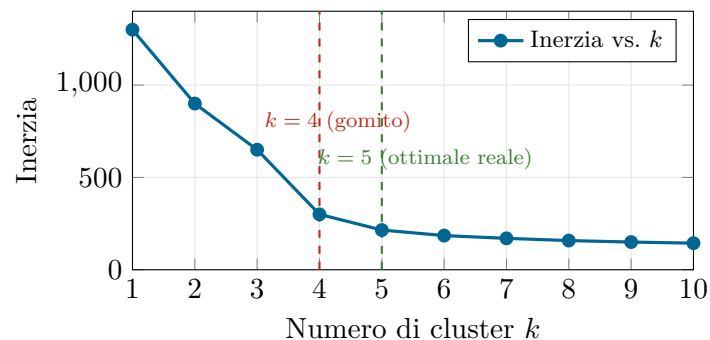
Per dataset molto grandi che non entrano in memoria, `MiniBatchKMeans` aggiorna i centroidi su **mini-batch** di istanze invece che sull'intero dataset. È circa 3–4 volte più veloce del K-Means standard, con un'inerzia leggermente superiore.

8.5 Scegliere il numero ottimale di cluster k

In un dataset reale (magari a 10 dimensioni) non è ovvio quanti centroidi creare. Si usano due metodi principali.

8.5.1 Metodo del Gomito (Elbow Method)

Si calcola l'**inerzia** per diversi valori di k e si traccia la curva. L'inerzia **diminuisce sempre** all'aumentare di k (più cluster $\rightarrow$ ogni punto è più vicino al proprio centroide). Si cerca il **punto di flesso** (il “gomito”) dove la curva inizia ad appiattirsi.



Limite del metodo del gomito

Il gomito è utile ma non sempre preciso: la curva può essere “arrotondata” senza un punto di flesso netto. Nel dataset a 5 blob, il gomito suggerisce $k = 4$ ma il valore corretto è $k = 5$. Serve sempre guardare anche il grafico dei dati e il coefficiente di Silhouette.

8.5.2 Coefficiente di Silhouette

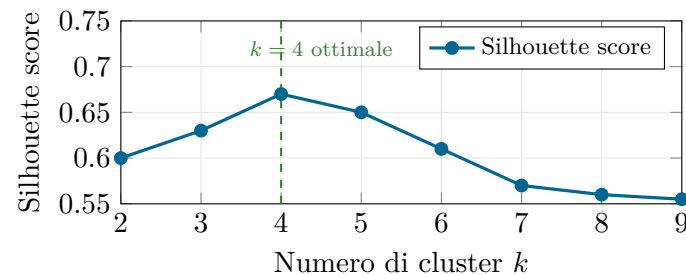
È una metrica più raffinata che valuta quanto ogni istanza è ben inserita nel proprio cluster. Per ogni istanza:

- a = distanza media dagli altri punti dello *stesso* cluster (intra-cluster).
- b = distanza media dai punti del *cluster più vicino* (inter-cluster, il secondo più vicino).

$$s = \frac{b - a}{\max(a, b)} \quad (8.1)$$

Valore di s	Interpretazione
$s \approx +1$	L'istanza è ben inserita nel suo cluster, lontana dagli altri – ottima clusterizzazione.
$s \approx 0$	L'istanza è vicina al confine tra due cluster – assegnazione incerta.
$s < 0$	$a > b$: l'istanza è stata assegnata al cluster sbagliato.

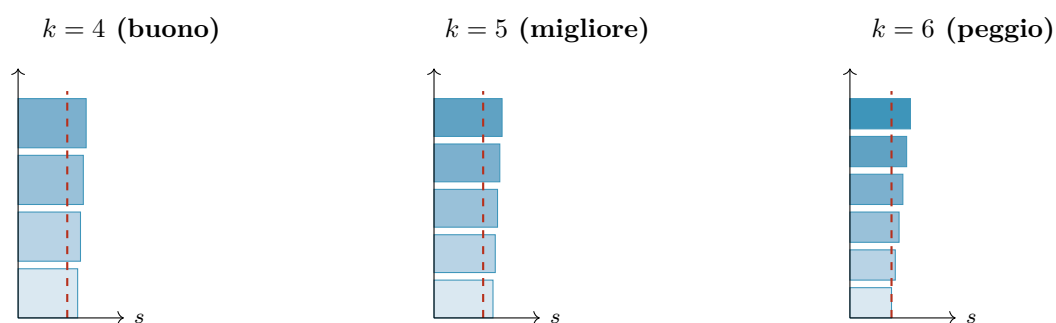
Il **silhouette score** globale è la media dei coefficienti di tutte le istanze. Si calcola con `silhouette_score(X, kmeans.labels_)`.



8.5.3 Diagramma a coltello (Silhouette Diagram)

Una visualizzazione ancora più informativa è il **silhouette diagram**: per ogni valore di k , si disegna un “coltello” per cluster. Ogni coltello mostra i coefficienti di silhouette ordinati in senso decrescente per tutte le istanze di quel cluster.

- L'**altezza** del coltello indica il numero di istanze nel cluster.
- La **larghezza** rappresenta il valore del coefficiente (più largo = meglio).
- La **linea tratteggiata verticale** rossa indica la media globale. Un buon clustering ha tutte le lame che superano questa linea e hanno dimensioni simili.



Come leggere il diagramma a coltello

Il clustering è **buono** se tutte le “lame” superano la linea rossa tratteggiata (media globale) e hanno dimensioni simili tra loro. Se alcune lame non raggiungono la media o sono molto più piccole delle altre, quei cluster sono “cattivi”.

Nel dataset a 5 blob: con $k = 4$, un cluster è molto più grande degli altri; con $k = 5$, tutti i cluster hanno dimensioni simili e valori di silhouette elevati. Quindi $k = 5$ è la scelta migliore, anche se il metodo del gomito suggeriva $k = 4$.

8.6 Applicazioni del K-Means

8.6.1 Segmentazione di immagini

K-Means può ridurre il numero di colori di un'immagine raggruppando i pixel per colore simile (*color segmentation*). Ogni pixel viene sostituito con il colore medio del suo cluster – utile per sistemi di object detection che hanno bisogno di semplificare la scena.

8.6.2 Apprendimento semi-supervisionato con K-Means

Una delle applicazioni più potenti del clustering: estrarre **istanze rappresentative** da un dataset non etichettato per minimizzare il lavoro di etichettatura manuale.

Digits dataset – da 74.8% a 90.9% con soli 50 label

Scenario: 1 400 immagini di cifre (dataset Digits), ma solo 50 possono essere etichettate manualmente.

Baseline (50 istanze casuali): Logistic Regression ottiene **74.8%** di accuratezza.

Con K-Means:

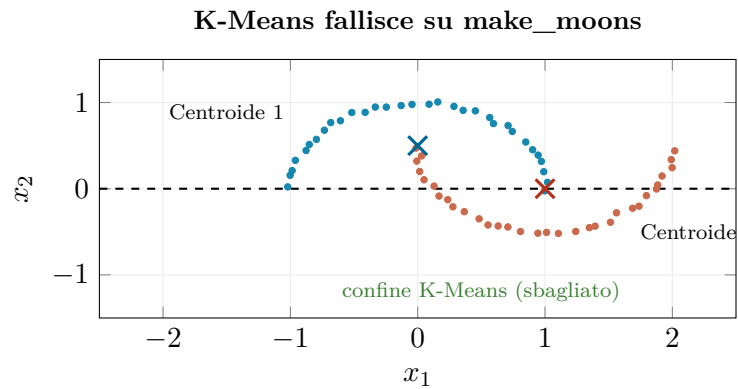
1. Clusterizzare le 1 400 immagini in 50 cluster.
2. Per ogni cluster, trovare l'immagine più vicina al centroide (*immagine rappresentativa*).
3. Etichettare solo queste 50 immagini rappresentative.
4. Addestrare Logistic Regression su 50 istanze rappresentative → **84.9%**.
5. **Propagare le etichette** a tutte le istanze dello stesso cluster → **89.4%**.
6. Eliminare l'1% più lontano dal centroide (outlier) → **90.9%**.

Con soli 50 label (meno di 5 per cifra in media) si raggiunge la stessa accuratezza del modello addestrato sull'intero dataset etichettato (90.7%). Le etichette propagate sono accurate al 97.5%.

8.7 Limiti del K-Means

Nonostante sia veloce e scalabile, K-Means ha limiti importanti:

- **Bisogna specificare k a priori:** non sempre è ovvio quanti cluster ci sono.
- **Convergenza a ottimi locali:** l'inizializzazione conta; K-Means++ e `n_init` riducono ma non eliminano il rischio.
- **Cluster sferici e isotropi:** K-Means lavora bene con “bolle” circolari (`make_blobs`), ma fallisce con:
 - cluster di **forma non sferica** (es. mezze lune, spirali – `make_moons`);
 - cluster di **densità o dimensioni diverse**;
 - cluster con **forma ellittica** orientata.
- **Sensibile alla scala:** le feature non scalate distorcono le distanze. **Sempre scalare i dati prima di K-Means.**



Non esiste una “pallottola d’argento”

Non esiste un algoritmo di clustering perfetto per ogni dataset. K-Means per le forme circolari, DBSCAN per le forme arbitrarie basate sulla densità (mezze lune, spirali), Gaussian Mixture Models per cluster ellissoidali. La scelta dipende dai dati.

Capitolo 9

DBSCAN, Isolation Forest e Anomaly Detection

Lezione 9 – 14/04/2026 – Géron, Cap. 9 (cont.)

9.1 DBSCAN: clustering basato sulla densità

A differenza del K-Means, che si basa sulle distanze dai centroidi, il **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*, Ester et al. 1996) definisce i cluster come **regioni ad alta densità separate da zone a bassa densità**. Non richiede di specificare k a priori.

9.1.1 Concetti chiave

Due iperparametri controllano tutto:

- ε (*epsilon*): raggio del vicinato – la distanza massima entro cui un punto è considerato “vicino”.
- `min_samples`: numero minimo di punti che devono cadere entro ε perché un punto sia considerato “denso”.

Ogni punto riceve una delle tre etichette:

Tipo	Definizione
Core Instance	Ha almeno <code>min_samples</code> punti nel raggio ε . È nel cuore di una regione densa.
Border Point	Non è core, ma cade entro ε di una core instance. È ai margini del cluster.
Noise / Outlier	Non è core e non è raggiungibile da nessuna core. Etichettato <code>-1</code> in Scikit-learn.

9.1.2 Funzionamento dell'algoritmo

1. Per ogni punto x , contare i vicini nel vicinato $B(x, \varepsilon)$.

2. Se il conteggio $\geq \text{min_samples}$, marcare x come *core instance*.
3. Espandere il cluster: partendo da una core instance, aggiungere tutti i punti *density-reachable* (raggiungibili attraverso una catena di core instances connesse).
4. I punti non raggiungibili da nessuna core instance sono *noise* – etichettati -1 .

Effetto di ε su `make_moons`

Con $\varepsilon = 0,05$ (troppo piccolo): l'algoritmo trova 7 cluster separati e molte anomalie – troppo restrittivo.

Con $\varepsilon = 0,2$ (giusto): trova esattamente 2 cluster (le due mezze lune) con pochissimi punti di rumore – perfetto per forme non lineari dove K-Means fallisce completamente.

9.1.3 Vantaggi e limiti

- **Pro:** cluster di qualsiasi forma (mezze lune, spirali, L-shape); gestisce nativamente il rumore (outlier = -1); non richiede di specificare k .
- **Contro:** fatica con densità molto variabili tra cluster; scala male in alta dimensione (*curse of dimensionality*); ε e `min_samples` vanno scelti con cura.

DBSCAN non ha `predict()`

Scikit-learn non fornisce un metodo `predict()` nativo per DBSCAN. Per classificare nuovi punti, si addestra un classificatore secondario (tipicamente K-NN) sulle core instances. Con una soglia di distanza si può anche rifiutare punti molto lontani come anomalie.

HDBSCAN (Hierarchical DBSCAN): variante che gestisce densità variabili costruendo una gerarchia di cluster – generalmente superiore a DBSCAN e disponibile nel pacchetto `scikit-learn-contrib`.

9.2 Cenno: Gaussian Mixture Models (GMM)

Il **Gaussian Mixture Model** è un modello probabilistico che assume che i dati siano stati generati da una miscela di k distribuzioni Gaussiane, ciascuna con il proprio centro μ_j e la propria forma Σ_j . A differenza del K-Means, il GMM assegna a ogni istanza una **probabilità di appartenenza** a ciascun cluster (*soft clustering*) e gestisce cluster di forma ellittica. I parametri si stimano con l'algoritmo EM (Expectation-Maximization). Per scegliere il numero di componenti si usano i criteri AIC e BIC. Il GMM è anche un **modello generativo**: può campionare nuovi dati dalla distribuzione appresa ed è usato per anomaly detection individuando istanze in regioni a bassa densità.

9.3 Isolation Forest: rilevazione delle anomalie

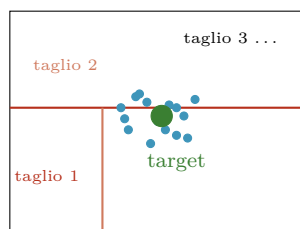
Isolation Forest (Liu, Ting & Zhou 2008) è un algoritmo specifico per l'*anomaly detection*, basato sul principio che le **anomalie sono poche e diverse**: sono più facili da isolare rispetto ai punti normali.

9.3.1 Logica fondamentale

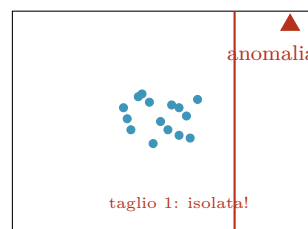
Invece di stimare densità o distanze, Isolation Forest misura **quanto rapidamente un punto viene isolato** da tagli casuali dello spazio delle feature.

- Un **punto normale** si trova in una regione densa → servono *molti* tagli casuali per isolarlo → si trova in foglie profonde dell'albero.
- Un'**anomalia** è lontana dalla massa dei dati → bastano *pochi* tagli → si trova vicino alla radice.

Punto normale – isolato tardi



Anomalia – isolata subito



9.3.2 Come funziona

1. Si costruisce una foresta di **isolation trees**: ogni albero seleziona una feature a caso, poi una soglia di taglio casuale tra il minimo e il massimo di quella feature nel nodo corrente. Si ricorre finché ogni punto è isolato.
2. Per ogni punto si calcola la **lunghezza media del percorso** su tutti gli alberi.
3. La lunghezza media viene **normalizzata** rispetto alla lunghezza attesa per un dataset casuale della stessa dimensione.
4. **Anomaly score**: più corto è il percorso medio, più anomalo è il punto.

9.3.3 Parametri fondamentali

- **n_estimators**: numero di alberi nella foresta (default: 100). Più alberi → meno varianza, ma più lento.
- **contamination**: proporzione attesa di anomalie nel dataset. Controlla direttamente la soglia di classificazione. Impostarlo è una **decisione di business**: quante anomalie mi aspetto? (es. 0.01 per 1% di frodi).
- **max_samples**: numero di istanze per albero (default: 'auto' = $\min(256, m)$). Sottocampionare aumenta la diversità tra alberi.
- **max_features**: numero di feature per albero; ridurlo aumenta la diversità.

9.3.4 Vantaggi e limiti

- **Vantaggi**: non calcola distanze né densità → computazionalmente efficiente; scala bene in alta dimensione; ottimo come baseline per anomaly detection.
- **Limiti**: fallisce se le anomalie formano cluster densi tra loro; i punteggi sono relativi alla distribuzione osservata; sensibile a feature irrilevanti.

Outlier detection vs. Novelty detection

Outlier detection: il training set può già contenere anomalie – il modello le cerca durante il training. È il caso tipico di Isolation Forest.

Novelty detection: il modello è addestrato su dati “puliti” (solo normali) e poi usato per rifiutare nuove osservazioni anomale. One-Class SVM è progettato per questo scenario.

9.4 Analisi esplorativa (EDA) e outlier

Prima di applicare qualsiasi modello di clustering o anomaly detection, è fondamentale capire la distribuzione dei dati tramite strumenti grafici.

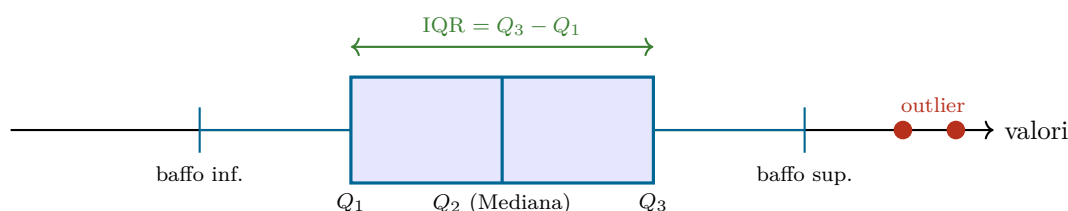
9.4.1 Istogrammi e il fenomeno del Capping

Analizzando gli attributi numerici di un dataset (es. dati immobiliari) si possono notare anomalie visive negli istogrammi:

- **Distribuzione attesa:** la mediana del reddito pro capite segue un andamento a campana.
- **Capping (limitazione):** valori oltre una certa soglia vengono accorpati in un unico bin (es. tutte le case con più di 51 anni classificate come “51 anni”). Crea un **picco anomalo** alla fine dell’istogramma che può confondere il modello – va trattato prima del training.

9.4.2 Il Boxplot (diagramma a scatola e baffi)

Il boxplot è lo strumento standard per visualizzare la dispersione di una variabile e identificare matematicamente gli outlier.



La **regola dei baffi** (regola di Tukey) per identificare gli outlier:

$$IQR = Q_3 - Q_1$$

$$\text{Soglia superiore} = Q_3 + 1,5 \times IQR \quad \text{Soglia inferiore} = Q_1 - 1,5 \times IQR$$

I punti che cadono oltre i baffi sono rappresentati come pallini separati e classificati come potenziali anomalie.

EDA pratica – come impostare contamination

Se un boxplot mostra che circa il 3% dei punti cade oltre i baffi, si può impostare `contamination=0.03` in Isolation Forest come stima iniziale. La scelta finale è una decisione di dominio: quanto costa un falso allarme rispetto a un'anomalia mancata?

9.5 Sintesi degli algoritmi non supervisionati

Algoritmo	Punto di forza	Uso ideale
K-Means	Veloce e semplice	Cluster sferici, k noto
Mini-Batch K-Means	Scalabilità estrema	Dataset enormi
DBSCAN	Forme arbitrarie, gestisce rumore	Cluster non lineari, densità uniforme
HDBSCAN	Densità variabili	Estensione robusta di DBSCAN
Agglomerative	Dendrogramma, no k fisso	Tassonomie, biologia
GMM	Soft, probabilistico, ellittico	Cluster ellissoidali
Isolation Forest	Isolamento rapido, scalabile	Anomaly detection, frodi
One-Class SVM	Frontiera non lineare	Novelty detection
LOF	Densità locale	Anomalie locali

Come scegliere l'algoritmo

- Cluster sferici, k noto, dataset grande: **K-Means** (o Mini-Batch).
- Forme arbitrarie, presenza di rumore: **DBSCAN** / **HDBSCAN**.
- Cluster ellittici, soft assignment, stima densità: **GMM**.
- Rilevazione anomalie scalabile, alta dimensione: **Isolation Forest**.
- Anomalie locali in bassa dimensione: **LOF**.

Non esiste una “pallottola d'argento”: la scelta dipende dalla forma dei dati, dalla dimensionalità e dall'obiettivo applicativo.

Capitolo 10

Reti Neurali Artificiali

Lezione 10 – 20/04/2026 – Géron, Cap. 10

10.1 Introduzione e applicazioni moderne

Le **Reti Neurali Artificiali** (ANN – *Artificial Neural Networks*) sono modelli computazionali ispirati al funzionamento del cervello biologico. Sono alla base delle applicazioni più potenti dell'IA moderna:

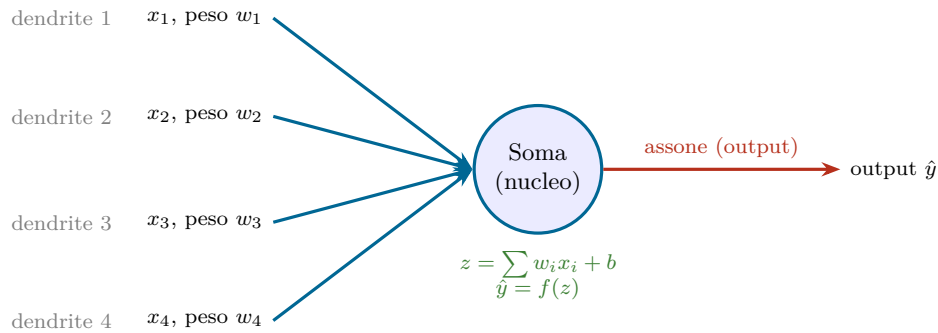
- **Visione artificiale:** riconoscimento di immagini, guida autonoma, diagnosi medica da immagini.
- **Linguaggio naturale:** traduzione automatica, chatbot, LLM (GPT, Claude).
- **Giochi:** AlphaGo e AlphaZero (DeepMind) hanno battuto i campioni mondiali di Go e scacchi.
- **Biologia:** AlphaFold ha predetto la struttura tridimensionale di quasi tutte le proteine conosciute.
- **Sistemi di raccomandazione:** Netflix, Spotify, Amazon.
- **Generative AI:** immagini (Stable Diffusion, DALL-E), testo, audio, video.

Il problema del Cold Start

Nei sistemi di raccomandazione, il *Cold Start* è la difficoltà di suggerire prodotti a **nuovi utenti** (o nuovi prodotti) per mancanza di dati storici. Se un utente è nuovo, non ha acquistato nulla: su cosa si basa il sistema? Soluzioni: chiedere preferenze iniziali, usare feature di contenuto (genere, artista), dati demografici o embedding pre-addestrati.

10.2 Dal neurone biologico al modello matematico

Il cervello umano contiene circa 10^{11} neuroni, ciascuno connesso in media a 10^4 altri neuroni. La comunicazione avviene attraverso impulsi elettrochimici (*potenziali d'azione*).



- **Dendriti** (input): ricevono i segnali dalle sinapsi – corrispondono alle feature x_i .
- **Soma** (elaborazione): calcola la somma pesata dei segnali in ingresso.
- **Assone** (output): trasmette il segnale se supera una soglia – corrisponde all’output $\hat{y}$.
- **Pesi sinaptici** w_i : la forza della connessione tra due neuroni. Idea di Hebb (1949): “*Neurons that fire together wire together*” – i neuroni che si attivano insieme rinforzano il loro legame.

10.2.1 McCulloch & Pitts (1943)

Il primo modello formale di neurone artificiale: input e output binari (0/1), soglia fissa. Dimostrarono che una rete finita di tali neuroni può calcolare qualsiasi **funzione booleana** – AND, OR, NOT e le loro combinazioni.

Calcoli logici con i neuroni

Si possono costruire le porte logiche fondamentali con neuroni a soglia. La soglia scatta quando $z \geq \theta$:

AND (2 input): pesi $w_1 = w_2 = 1$, soglia $\theta = 2$. Output 1 solo se *entrambi* gli input sono 1: $z = 1 + 1 = 2 \geq 2$ ✓.

OR (2 input): pesi $w_1 = w_2 = 1$, soglia $\theta = 1$. Output 1 se *almeno uno* è 1: $z = 1 \geq 1$ ✓.

NOT (1 input): peso $w_1 = -1$, bias $b = 1$. Output 1 se input è 0: $z = -0 + 1 = 1 \geq 1$ ✓.

Combinando AND, OR e NOT è possibile costruire qualsiasi circuito logico – in teoria, un cervello artificiale.

10.3 Il Percettrone

Frank Rosenblatt (1957) generalizza il neurone McCulloch-Pitts introducendo i **pesi apprendibili**: anziché fissare w_i manualmente, il percettrone li impara dai dati.

10.3.1 Matematica del percettrone

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b \quad (10.1)$$

Perché la trasposta $\mathbf{w}^T$?

Per convenzione $\mathbf{w}$ e $\mathbf{x}$ sono vettori colonna. Il prodotto scalare si calcola moltiplicando riga per colonna, quindi si traspone il vettore pesi: $\mathbf{w}^T \mathbf{x} = \sum_i w_i x_i$. È solo notazione – il risultato è lo stesso.

10.3.2 Funzione di attivazione Heaviside

Il perceptrone classico usa la **funzione gradino di Heaviside**:

$$H(z) = \begin{cases} 0 & \text{se } z < 0 \\ 1 & \text{se } z \geq 0 \end{cases} \quad (10.2)$$

L'output è binario: il neurone *spara* (1) se il segnale totale supera la soglia, altrimenti rimane silenzioso (0).

Analogia sensoriale

Udito in un bar affollato: il rumore di fondo costante non supera la soglia neuronale – viene filtrato. Se qualcuno urla il tuo nome, il peso w_i associato al suono del tuo nome è alto: $z \geq 0$ e il neurone “spara”, facendoti girare la testa.

Vista: una luce ambiente costante non attiva i fotorecettori di risposta al movimento. Una luce stroboscopica sì – è il cambiamento rapido che supera la soglia.

10.3.3 Regola di apprendimento di Hebb

Il perceptrone aggiorna i pesi in base all'errore – quanto la previsione si discosta dal target reale:

$$w_i^{(\text{next})} = w_i + \eta \cdot (y - \hat{y}) \cdot x_i \quad (10.3)$$

dove η è il **learning rate** (tasso di apprendimento) – un iperparametro che controlla quanto è grande ogni aggiornamento.

- Se $\hat{y} = y$ (previsione corretta): errore = 0, nessun aggiornamento.
- Se $\hat{y} < y$ (sotto-stima): i pesi per gli input positivi aumentano.
- Se $\hat{y} > y$ (sovra-stima): i pesi per gli input positivi diminuiscono.

Teorema di convergenza di Rosenblatt (1958): se i dati sono **linearmente separabili**, l'algoritmo converge in un numero finito di passi – troverà sempre una soluzione.

10.3.4 Limiti del perceptrone e l'Inverno dell'IA

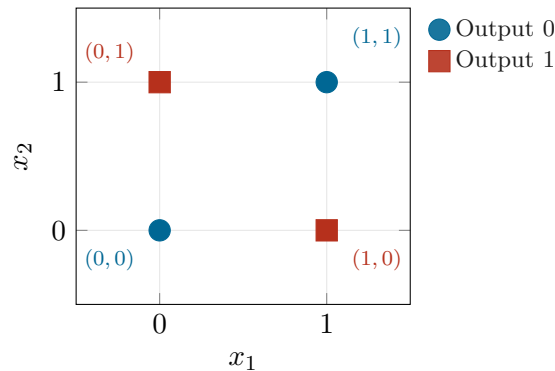
Il perceptrone classico risolve solo problemi **linearmente separabili** – quelli in cui esiste una retta (o un iperpiano) che separa le due classi.

Il problema dello XOR

Nel libro *Perceptrons* (1969), Minsky e Papert dimostrarono che un singolo percettrone **non può** risolvere la funzione XOR: le classi $\{(0,0), (1,1)\}$ (output 0) e $\{(0,1), (1,0)\}$ (output 1) non sono separabili da nessuna retta.

Questo risultato – amplificato oltre misura – causò il **Primo Inverno dell'IA**: finanziamenti tagliati, interesse crollato, ricerca bloccata per un decennio.

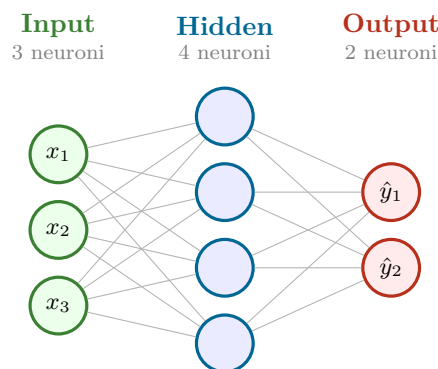
XOR – non linearmente separabile



La soluzione è aggiungere **strati nascosti** – i *Multi-Layer Perceptron* possono risolvere XOR e qualsiasi altra funzione.

10.4 Architettura del Multi-Layer Perceptron (MLP)

Un **MLP** è una rete feed-forward (il segnale fluisce solo in avanti) con uno o più **hidden layer** tra input e output.



Dense Layer (o *Fully Connected*): ogni neurone è connesso a *tutti* i neuroni dello strato precedente. In forma matriciale:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b} \quad \mathbf{X} \in \mathbb{R}^{m \times n}, \quad \mathbf{W} \in \mathbb{R}^{n \times h}, \quad \mathbf{b} \in \mathbb{R}^h$$

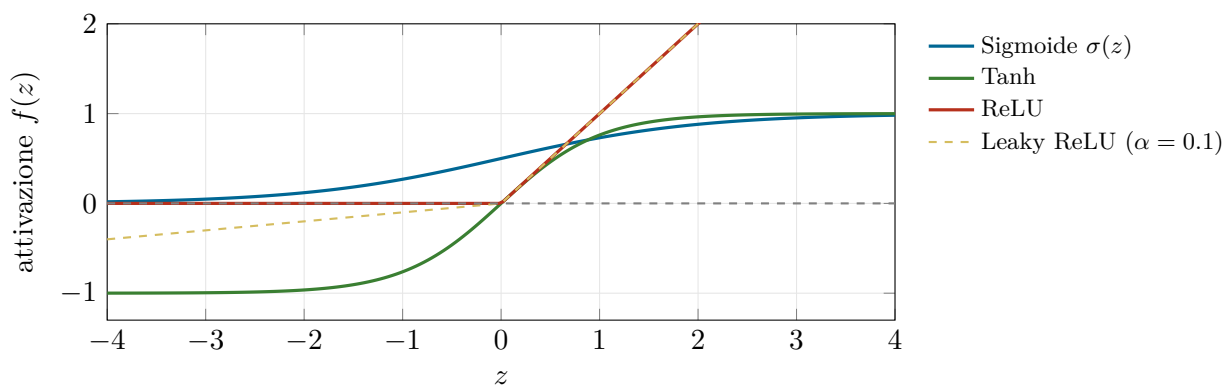
Perché GPU e TPU?

Il calcolo dei layer densi si riduce a moltiplicazioni matriciali – operazioni altamente parallelizzabili. Le GPU (originalmente progettate per rendering 3D) e le TPU (Tensor Processing Units, Google) eseguono migliaia di queste operazioni contemporaneamente: una rete che su CPU richiederebbe ore si addestra in minuti.

Al contrario, algoritmi *sequenziali* come il calcolo di Fibonacci (ogni passo dipende dai due precedenti) non beneficiano della parallelizzazione.

10.5 Funzioni di attivazione

Senza funzioni di attivazione non lineari, un MLP con più layer sarebbe equivalente a un singolo layer lineare – inutile. Le funzioni di attivazione **introducono la non-linearità** che permette alla rete di approssimare funzioni complesse.



- **Sigmoidale** $\sigma(z) = \frac{1}{1+e^{-z}}$: output in $(0, 1)$. Utile nell'output per classificazione binaria. Problema: **saturazione** – per z molto grandi o piccoli, il gradiente tende a zero (*vanishing gradient*).
- **Tanh**: output in $(-1, 1)$, centrata in zero – convergenza più veloce della sigmoide. Stesso problema di saturazione.
- **ReLU** $\max(0, z)$: standard de facto per i layer nascosti. Veloce da calcolare, nessuna saturazione per $z > 0$. Problema: *dying ReLU* – neuroni con $z < 0$ hanno gradiente sempre zero e smettono di imparare.
- **Leaky ReLU**: $\max(\alpha z, z)$ con α piccolo (es. 0.1). Risolve il dying ReLU mantenendo un piccolo gradiente per $z < 0$.

10.6 Backpropagation

L'algoritmo **backpropagation** (Rumelhart, Hinton, Williams 1986) è il motore dell'addestramento delle reti neurali: calcola efficientemente il gradiente della funzione di costo L rispetto a *tutti* i pesi.

10.6.1 Le quattro fasi

1. **Forward pass:** un mini-batch di dati attraversa la rete strato per strato; si calcolano tutte le attivazioni $\mathbf{a}^{(l)}$ e infine l'output $\hat{\mathbf{y}}$.
2. **Calcolo della loss:** si confronta $\hat{\mathbf{y}}$ con il target $\mathbf{y}$ tramite la funzione di costo (es. MSE, Cross-Entropy).
3. **Backward pass:** si applica la **regola della catena** a ritroso, propagando il gradiente da output a input:

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)}$$

dove $\delta_j^{(l)}$ è il “segnale di errore” del neurone j al layer l .

4. **Aggiornamento dei pesi** con gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L$$

Backpropagation non è magia

La backpropagation è semplicemente una **applicazione efficiente della regola della catena** del calcolo differenziale. Il nome “back” viene dal fatto che i gradienti fluiscono all'indietro – dall'output all'input – mentre i dati nel forward pass fluiscono in avanti. L'efficienza viene dal riutilizzare i calcoli intermedi già fatti nel forward pass.

10.7 Regressione e classificazione con MLP

L'output layer si configura in base al tipo di problema. Tutto il resto della rete (layer nascosti) usa tipicamente ReLU.

Problema	Output layer	Funzione di costo
Regressione (1 valore)	1 neurone lineare	MSE o Huber loss
Regressione (K valori)	K neuroni lineari	MSE / Huber
Classificazione binaria	1 neurone + Sigmoidale	Binary Cross-Entropy
Classificazione multilabel	K neuroni + Sigmoidale	Binary Cross-Entropy
Classificazione multiclasse	K neuroni + Softmax	Categorical Cross-Entropy

Softmax

La funzione **Softmax** trasforma un vettore di logit (punteggi grezzi) in una distribuzione di probabilità:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \sum_{k=1}^K \text{softmax}(\mathbf{z})_k = 1$$

Usata nell'output layer multiclasse: garantisce che le probabilità siano tutte positive e sommino a 1. La classe predetta è quella con probabilità massima.

Classificazione su MNIST con Keras

La rete ha $784 \times 300 + 300 = 235\,500$ parametri nel primo layer, $300 \times 100 + 100 = 30\,100$ nel secondo, $100 \times 10 + 10 = 1\,010$ nel terzo – totale $\approx 267\,000$ parametri addestrabili.

10.8 Fine-tuning degli iperparametri

Un MLP ha molti iperparametri da scegliere. Géron suggerisce alcune regole pratiche:

- **Numero di layer nascosti:** iniziare con 1–2 layer (sufficienti per la maggior parte dei problemi). Aggiungere profondità solo se il modello è insufficiente – le reti profonde imparano gerarchie di feature (bordi $\rightarrow$ texture $\rightarrow$ parti $\rightarrow$ oggetti).
- **Numero di neuroni per layer:** tipicamente tra 32 e 512, potenze di 2 per compatibilità GPU. Conviene iniziare largo e poi ridurre.
- **Learning rate η :** l'iperparametro più importante da sintonizzare. Troppo alto: instabilità. Troppo basso: convergenza lentissima. Valore tipico: 10^{-3} con Adam.
- **Batch size:** 32–256. Batch grandi convergono più velocemente ma generalizzano peggio.
- **Ottimizzatore:** Adam è il default moderno (adatta automaticamente il learning rate per ogni parametro).

Metodi di ricerca degli iperparametri:

- **Grid search:** prova ogni combinazione di una griglia predefinita. Esaustivo ma costoso.
- **Random search:** campiona combinazioni casuali – spesso più efficiente della grid search.
- **Bayesian optimization:** usa un modello probabilistico per guidare la ricerca (Optuna, Keras Tuner, Hyperopt).

10.9 Teorema di approssimazione universale

Cybenko (1989) – Hornik (1991)

Un MLP con un **singolo hidden layer** di larghezza sufficiente e funzione di attivazione non polinomiale può approssimare qualsiasi funzione continua su un insieme compatto, con precisione arbitraria.

In altre parole: in teoria, una rete neurale può imparare *qualsiasi cosa*. In pratica, però:

- Reti **profonde** (molti layer) sono più efficienti delle reti **larghe** (un solo layer enorme): imparano gerarchie di feature con meno parametri totali.
- Più profondità $\rightarrow$ nuovi problemi: **vanishing gradient** e training instabile. Questo è l'argomento della prossima lezione.

Capitolo 11

Addestrare Reti Neurali Profonde

Lezione 11 – 27/04/2026 – Géron, Cap. 11

11.1 Problemi nell'addestramento di reti profonde

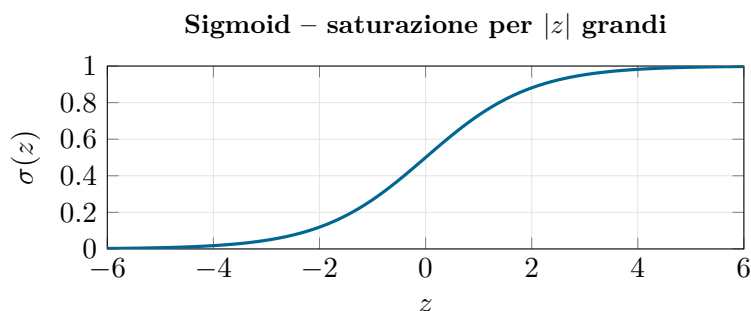
Addestrare una rete neurale profonda (DNN) con molti layer nascosti non è semplice. I problemi principali sono:

- **Gradienti instabili:** i gradienti possono diventare molto piccoli (*vanishing gradient*) o molto grandi (*exploding gradient*) durante la backpropagation, rendendo difficoltoso l'addestramento dei layer inferiori.
- **Dati insufficienti:** modelli con milioni di parametri richiedono enormi quantità di dati etichettati per evitare l'overfitting.
- **Tempistiche di training:** l'addestramento può essere estremamente lento.

11.2 Il problema del vanishing/exploding gradient

Durante la backpropagation, l'errore viene propagato dall'output verso l'input. Spesso i gradienti diventano sempre più piccoli man mano che si procede verso i layer inferiori (**vanishing gradient**). In alcuni casi, accade il contrario: i gradienti crescono in modo esplosivo (**exploding gradient**), portando a aggiornamenti dei pesi instabili e alla divergenza dell'addestramento.

Uno dei motivi principali è la scelta della funzione di attivazione **sigmoide**, che satura per valori grandi (positivi o negativi), producendo gradienti prossimi a zero.



11.3 Inizializzazione di Glorot e He

Una soluzione fondamentale è un'adeguata **inizializzazione dei pesi**. Glorot e Bengio (2010) proposero di inizializzare i pesi in modo che la varianza degli output di ogni layer sia uguale alla varianza dei suoi input, e lo stesso per i gradienti nella backpropagation.

Inizializzazione di Glorot (o Xavier)

Per le funzioni di attivazione sigmoide, tanh e softmax, si usa una distribuzione normale con varianza

$$\sigma^2 = \frac{1}{\text{fan}_{\text{avg}}}, \quad \text{fan}_{\text{avg}} = \frac{\text{fan}_{\text{in}} + \text{fan}_{\text{out}}}{2}$$

o una distribuzione uniforme nell'intervallo $\left[-\sqrt{\frac{3}{\text{fan}_{\text{avg}}}}, +\sqrt{\frac{3}{\text{fan}_{\text{avg}}}}\right]$.

Inizializzazione di He (Kaiming)

Per ReLU e le sue varianti (Leaky ReLU, ELU, ecc.), si usa una varianza doppia:

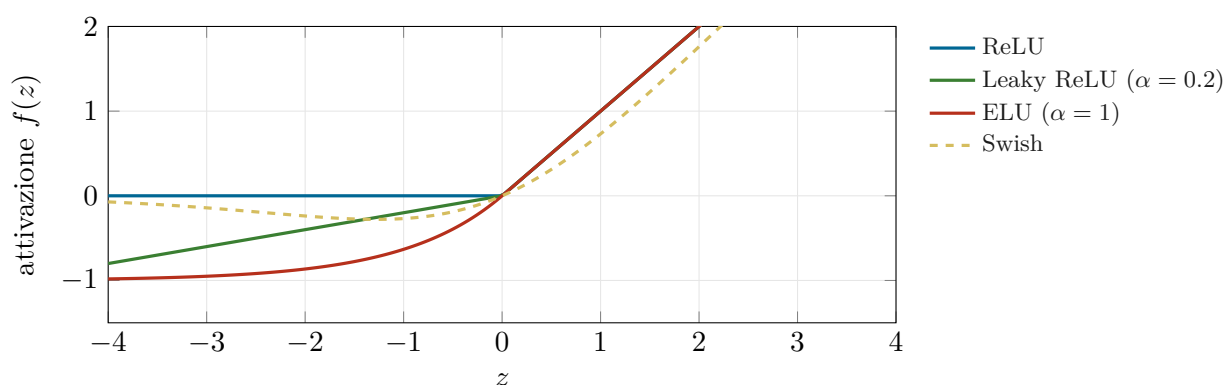
$$\sigma^2 = \frac{2}{\text{fan}_{\text{in}}}$$

In Keras, è sufficiente specificare il parametro `kernel_initializer`:

11.4 Funzioni di attivazione moderne

Le funzioni di attivazione influenzano notevolmente la stabilità dei gradienti. Ecco le più comuni:

- **ReLU** ($\max(0, z)$): veloce e non satura per $z > 0$, ma soffre del problema dei “neuroni morenti” (dying ReLU) quando l'input è sempre negativo.
- **Leaky ReLU** ($\max(\alpha z, z)$): introduce una piccola pendenza α per $z < 0$ (es. 0.2), risolvendo il problema dei neuroni morenti.
- **ELU**: liscia e con media vicina a zero, aiuta la convergenza ma è più costosa da calcolare.
- **SELU**: versione scalata di ELU che auto-normalizza l'output, ma richiede condizioni specifiche (inizializzazione LeCun, input standardizzati, no dropout standard).
- **GELU, Swish, Mish**: funzioni moderne, lisce e non monotoniche, che spesso superano ReLU su compiti complessi.



Quale attivazione scegliere?

- **ReLU** è un buon default per compiti semplici: veloce e ben supportato.
- **Swish** o **GELU** sono ottimi per compiti complessi.
- **Leaky ReLU** è utile se si osservano molti neuroni morti.
- **SELU** può funzionare molto bene per MLP profondi, ma attenzione ai vincoli.

11.5 Gradient Clipping

Per il problema dell'**exploding gradient**, una soluzione efficace è il **gradient clipping**: si limitano i gradienti a un valore massimo (o norma massima) prima dell'aggiornamento dei pesi. Questa tecnica è particolarmente utile nelle **reti ricorrenti** (RNN).

11.6 Ottimizzatori: Momentum

Lo SGD classico può essere lento. Lo **ottimizzatore con momentum** accelera la convergenza aggiungendo una frazione dell'aggiornamento precedente all'aggiornamento corrente.

11.6.1 Momentum

$$\begin{cases} \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta) \\ \theta \leftarrow \theta + \mathbf{m} \end{cases} \quad (11.1)$$

dove β è il **momentum** (tipicamente 0.9). L'aggiornamento accumula velocità nelle direzioni di gradiente costante, accelerando la convergenza.

11.6.2 Nesterov Accelerated Gradient (NAG)

Una variante che calcola il gradiente non nella posizione attuale θ , ma in $\theta + \beta \mathbf{m}$ (una “previsione” della prossima posizione). Questo riduce le oscillazioni e converge più velocemente.

Adam

Adam combina momentum e adattamento del learning rate ed è spesso il primo ottimizzatore da provare. AdamW è una variante con weight decay corretto.

11.7 Tecniche di regolarizzazione per evitare overfitting

Le reti profonde hanno molti parametri e tendono a overfittare. Ecco le tecniche principali.

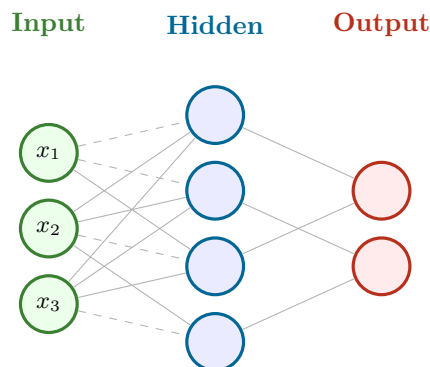
11.7.1 Regolarizzazione ℓ_1 e ℓ_2

Si aggiunge un termine di penalità alla funzione di costo:

- ℓ_2 (Ridge): $\frac{\alpha}{2} \sum \mathbf{w}^2$ (pesi piccoli)
- ℓ_1 (Lasso): $\alpha \sum |\mathbf{w}|$ (pesi sparsi)

11.7.2 Dropout

Ad ogni passo di training, ogni neurone (esclusi quelli di output) ha una probabilità p di essere “droppato” (ignorato). Questo forza la rete a non dipendere troppo da singoli neuroni, migliorando la generalizzazione. La probabilità di dropout è tipicamente tra il 10% e il 50%.



11.7.3 Max-norm regularization

Vincola la norma ℓ_2 dei pesi in ingresso a ogni neurone a essere $\leq r$. Dopo ogni aggiornamento, se $\|\mathbf{w}\|_2 > r$, si riscalda: $\mathbf{w} \leftarrow \mathbf{w} \frac{r}{\|\mathbf{w}\|_2}$.

Regolarizzazione: consigli pratici

- **Early stopping:** è spesso la tecnica più efficace e semplice.
- **Batch normalization:** regolarizza naturalmente.
- **Dropout:** molto efficace, specialmente per reti grandi (con tassi tra 0.2 e 0.5).
- **Max-norm:** utile in combinazione con dropout.
- ℓ_2 : ok con SGD, ma con Adam usare AdamW.

11.8 Tecniche per reti profonde

Problema	Tecnica
Vanishing/Exploding gradient	Glorot/He initialization, Batch Normalization, Gradient Clipping
Lenta convergenza	Momentum, Nesterov, Adam
Overfitting	Dropout, ℓ_1/ℓ_2 reg., Max-norm, Early Stopping
Scarsa generalizzazione	Data augmentation, Transfer learning

.1 FORMULE

Regressione lineare	$y = \theta_0 + \theta_1 x$
Accuratezza	$\frac{TP + TN}{TP + TN + FP + FN}$
Precisione	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F1-Score	$2 \cdot \frac{P \cdot R}{P + R}$
False Positive Rate	$\frac{FP}{FP + TN}$
Impurità di Gini	$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$
Entropia di Shannon	$H_i = - \sum_{k=1}^n p_{i,k} \log_2 p_{i,k}$
Costo CART (classif.)	$J(k, t_k) = \frac{m_L}{m} G_L + \frac{m_R}{m} G_R$
Costo CART (regr.)	$J(k, t_k) = \frac{m_L}{m} MSE_L + \frac{m_R}{m} MSE_R$
OvO classificatori	$\frac{N(N-1)}{2}$
Inerzia (K-Means)	$J = \sum_j \sum_{x \in C_j} \ x - \mu_j\ ^2$
Silhouette	$s = \frac{b - a}{\max(a, b)}$
Percettrone	$z = \mathbf{w}^T \mathbf{x} + b$
Softmax	$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$
Aggiornamento Hebb	$w \leftarrow w + \eta(y - \hat{y}) x$
Init. Glorot	$\sigma^2 = \frac{2}{n_{in} + n_{out}}$
Init. He	$\sigma^2 = \frac{2}{n_{in}}$